NANYANG TECHNOLOGICAL UNIVERSITY

Enabling High Level Design of Adaptive Systems with Partial Reconfiguration

by

Vipin Kizheppatt

Confirmation Report Submitted to the School of Computer Engineering of Nanyang Technological University for Confirmation for Admission to the Degree of Doctor of Philosophy

Supervised by

Asst. Prof. Suhaib A Fahmy

January 2012

Contents

List of Figures

List of Tables

Acknowledgements

First and foremost I would like to thank my mentor and supervisor Suhaib Fahmy for selecting me as his first PhD student and giving an opportunity to work in this vibrant university. His charm and enthusiasm have always helped me to perform my research work under a relaxed and energetic environment. His guidance really helped me to understand the research strategies under an academic environment. I appreciate his ideas, suggestions, and especially the time he has taken to help me correct mistakes and improve my writing. I am really thankful to my friends and colleagues at the Centre for High Performance Embedded Systems (CHiPES) for their support and encouragement. Chua Ngee Tat, laboratory executive at CHiPES, was always ready to lend a helping hand whenever I faced software related issues. I would also like to thank Associate Professor Vinod A Prasad, School of Computer Engineering, NTU and Associate Professor Vincent John Mooney, School of Electrical and Computer Engineering, Georgia Institute of Technology for their encouragement and support during the courses they taught me. I am taking this opportunity to thank my previous employer Processor Systems India Pvt. Ltd, Bangalore, India for providing me an opportunity work in a competitive industrial scenario. My interest in FPGAs was stimulated during my employment and I would like to thank my former mentors Manjusha S and Vinod N and my former manager Jaison T D for their guidance on FPGA based systems design and industry standards, which were proven to be invaluable during my research. Finally I like to thank my parents for their constant love and encouragement. I am indebted to the prayers, pain, and efforts they are taking for supporting me in pursuing higher studies. I bow to those invisible hands, which have brought me where I am today and pray to that ultimate teacher for guiding my efforts to achieve its ultimate goal.

Abstract

Adaptive systems have the ability to respond to environmental conditions, by modifying their processing at runtime. While this is easy to do in software systems, modern algorithms can be computationally expensive, requiring powerful processors. At the same time hardware is not as flexible. Field programmable gate arrays (FPGAs) are recognised as being suitable for adaptive systems implementation, due to their flexibility and high performance. The use of partial reconfiguration on FPGAs to implement adaptive systems has been proposed many times in the literature. However the design process for partially reconfigurable systems is complex and requires specialist knowledge on behalf of the application designer. Hence, it has remained a rarely used capability outside of academic circles. We propose a new approach to leverage partial reconfiguration within adaptive systems, by integrating with, rather than circumventing, supported vendor tool flows, while automating many of the steps that have made such designs more difficult in the past. This makes it possible for system designers with less FPGA expertise to use partial reconfiguration when designing adaptive systems.

Chapter 1

Introduction

Rapid advancements in technology and constantly evolving standards are pushing system designers to the limits. Development time for newer standard specifications is continuously reducing, which demands frequent system upgrades. Several latest standards require complex data processing capabilities and as well as high data rates. Software modifications alone cannot keep up with this pace, which call for changes at hardware level. Developing specialised chips (ASICs) for these evolving standards is becoming less and less practical due to the long turnaround time required for ASIC development and the very high cost associated with the integrated circuit development. Reconfigurable computing is a promising solution for this challenge. Reconfigurable computing makes it possible to bring some flexibility into hardware after system deployment by using high performance computing fabrics such as Field Programmable Gate Arrays (FPGAs). Reconfigurable computing tries to combine the high performance of hardware with the flexibility of software.

Practically, a single hardware platform can be used for implementing multiple circuits using reconfiguration. For example, a hardware module, which can be used for implementing audio filters during music playback, can be used for implementing video decoders while the system plays a movie. These hardware modifications will be transparent to the end user and the necessary circuitry is automatically loaded. The advantages of using such a platform are multifaceted. The cost can be considerably reduced along with the size and weight of the system as well as the power consumption.

Another greater advantage is upgradability. Once better hardware architecture is available, the system can be instantly upgraded with minimal cost and without any component level hardware modifications.

1.1 Adaptive Systems

Adaptive systems are the systems which can evolve their operations based on their surroundings. They have the ability to respond to environmental conditions, by modifying their processing at runtime. This adaptability can lead to more sophisticated applications as well as improved performance. For example, a driver assistance system can modify its analysis algorithms based on road conditions [2] and a software defined radio can modify its modulation scheme based on channel conditions [3]. While this is easy to do in software systems, modern algorithms can be computationally expensive, requiring powerful processors. At the same time hardware is not as flexible. Reconfigurable computing techniques find extensive applications in adaptive systems implementations.

In recent years, the research interest in the adaptive systems field is increasing in order to achieve better system performance while meeting the environmental constrains. The development of cognitive radio is a classical example for this [4]. This invention was motivated by the fact that, although the available radio spectrum for future communications is quite limited, the present allocation of the spectrum is heavily underused. In order to improve the system efficiency, a new radio technology had to be developed, which must be highly adaptive in nature, capable of modifying several processing aspects such as coding scheme, modulation technique used etc. during runtime. This adaptability requires the system to be more dynamic in nature and able to change its processing strategy at runtime. This adaptability cannot be completely captured by software techniques. Sufficient hardware support needs to be provided to the system to achieve the desired performance.

An important challenge faced by adaptive systems design is the level of abstraction. Most of the design methodologies currently followed are not at system level. The

system designer is expected have knowledge about the lower hardware level intricacies. This means that adaptive systems are often designed using an ad-hoc approach, where the system design and implementation are tightly coupled. This makes the design complex and hard to modify. This is a major hurdle in leveraging adaptive systems as a widely accepted design practice like embedded systems development. In the subsequent sections we will see how FPGAs have contributed in adaptive systems development through recent improvement in design techniques.

1.2 FPGAs as Adaptive Hardware Platform

The invention of Field Programmable Gate Arrays (FPGAs) fuelled the developments in reconfigurable computing. FPGAs started as simple programmable chips, mainly used for glue logic implementations, and grew to fully-fledge programmable chips capable of implementing complete systems [5], thanks to the integration of built in hard-macros such as embedded processors, DSP blocks and BlockRAMs. As the name indicates, the greatest advantage of FPGAs is their on-site programmability. Design errors detected at a later stage can be corrected simply by configuring the FPGA using a new bitstream. Similarly, updates to the original design can be made in-field. FPGAs achieve their programmability by changing the contents of their configuration memory. Configuration memory holds the bitstream, which configures the logic and its routing implemented in the FPGA. These bitstreams are generated from the designer specification, usually in an HDL form, with the help of vendor-supported tool chain.

The main building block of FPGA logic is the lookup table (LUT). A LUT is a small memory usually 1 bit wide and 16 or 64 bits deep. By storing appropriate values in these memory locations, any Boolean function can be implemented. FPGAs also contain programmable routing resources and switch boxes, which make it possible to connect logic in a highly flexible manner. Dedicated routing resources are available for critical signals such as clocks and reset. The inherent parallel processing capability of hardware helps in achieving very high throughput. Another advantage of FPGAs is their programmable I/O pins. This makes them suitable for interfacing with a

variety of peripherals using different I/O standards. In recent years, FPGAs have been able to successfully challenge dedicated hardware (ASIC) implementation of several systems. This is mainly attributed to their reprogrammability, increasing logic density, and decreasing cost and power consumption. For moderate production runs FPGAs prove to be more cost effective compared to ASICs due to the very high non-recurring engineering (NRE) cost associated with integrated circuit manufacturing processes.

FPGAs are highly suitable for adaptive systems implementation. Their reprogrammability is one of the fundamental requirements in adaptive systems implementation. Moreover, recent FPGAs come with built in processors, which are capable of running software which is useful in adaptive systems. Extensive vendor-supported tools and intellectual property (IP) availability make FPGAs the preferred choice of adaptive system designers.

Although latest FPGAs provide substantial amount of logic resources sufficient for several systems implementations, for many FPGA based systems implementation designers face resource crunches. The cost of FPGA chips increases with their logic capacity, and using larger FPGAs for designs increases system cost. Recent advancements in FPGA design approaches help in overcoming this issue.

1.3 Partial Reconfiguration

The process of altering the logic implemented in an FPGA by means of loading a new bitstream is known as configuration. Traditionally during a configuration operation, the FPGA is not available for the user logic to execute. The configuration time can range from several micro-seconds to a few milli-seconds depending upon the FPGA logic density. This delay in reconfiguration makes FPGAs unsuitable for several real-time adaptive systems implementation. This has lead to the development of Partial Reconfiguration (PR). PR is a technique supported by new generation FPGAs especially from Xilinx Inc. PR has remained a constant research theme within the FPGA community since it was first mooted. PR takes advantage of the fact that FPGA bitstreams are stored in configuration memory and by selectively modifying

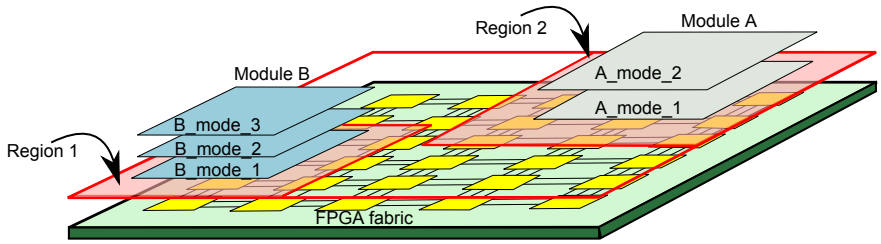


Figure 1.1: Partial Reconfiguration.

the contents of the configuration memory, portions of the design can be altered, while the remaining portions continue to operate without interruption. PR has made FPGAs more suitable for adaptive systems implementation, since the whole system doesn't need to be stopped in order to modify the functionality of some portions of the system. PR brings an additional dimension to FPGA operation, time. On the same FPGA fabric, there can be different functional units at different time instances as shown in Fig. 1.1. The time when a new functional unit is loaded depends upon different system parameters. For adaptive systems, these parameters will be the variations in the system environment.

PR brings several advantages other than making the system more flexible, such as

- The logic capacity of the FPGA is effectively increased, since several processing units can use the same FPGA resources at different time instances.
- For several applications, portions of the design remain inactive for long periods during system operation. Nevertheless, this logic consumes power. Although techniques such as clock gating can reduce power consumption, loading the inactive region with a blank bitstream helps in reducing the power consumption to a much lower level [6].
- Using PR, a smaller FPGA can be used for implementing larger designs using time-multiplexing. This helps in reducing the overall system cost.

1.4 Motivations

The increasing demand for adaptive systems, and at the same time the lack of versatile tools for their implementation is our primary motive for this research. Although PR based FPGA designs are highly suitable for adaptive systems implementation, presently available tools are neither mature enough nor sufficiently automated. In vendor-supported PR tool flows, the designer has to provide several manual inputs. The efficiency of system implementation greatly depends upon these inputs. These inputs are generally target specific FPGA architecture. This requires the system designer to have expertise in FPGA architectures. Similarly in order to optimise the design, the designer has to know the low level operations performed during PR. Such an ad-hoc, manual design process is highly time consuming and generally leads to sub-optimal results. Target architecture dependency makes PR an expert feature and makes it less attractive to system level designers. We feel that the level of abstraction in PR-based adaptive system needs to be increased to a functional level and only minimal architecture-dependent features should be exposed to the system level designer.

Another important limitation of present PR based systems is the run-time management of adaptive systems. The particular configurations the FPGA will operate under different environmental conditions have to be explicitly coded by the system designer. This code includes the information about specific bitstreams which should to configure the FPGA under different circumstances. This design paradigm needs to be changed. The configuration management operation should be abstracted to a higher level, where the system designer can specify it using a high level language. Automated tools should be able to determine lower level details such as the bitstreams that needs to be configured. The level of configuration, such as functional, structural, parametric, etc., needs to be abstracted away from the system designer.

Although vendors provide several IP cores to FPGA system designers, there is a shortage of library components targeted at adaptive system design. There should be application dependent, configurable functional unit libraries, which can be used by system designers to easily build their systems. The availability of pre-built functional

units as well as automated tools will make PR based adaptive systems more appealing for system designers.

1.5 Objectives

The main objectives of the research work are to:

1. Determine how adaptive systems can be described in a way that can be mapped to real implementation.
2. Demonstrate suitability of FPGA PR for adaptive systems.
3. Determine the metrics to be optimised, e.g. reconfiguration time.
4. Develop techniques and tools to automate the PR design process including partitioning, synthesis, and floorplanning, optimising for PR performance.
5. Develop an abstraction layer to assist design-time and run-time processes and management of PR systems.
6. Develop an IP library for adaptive PR systems and demonstrate the above framework on real applications.

1.6 Current Contributions

1. We have performed a comprehensive study of the partial reconfiguration process, from both the tools and architecture perspective.
2. Detailed architecture study of PR capable FPGAs and identified the metrics associated with PR as well as the limitations of current PR design flows.
3. An efficient partitioning algorithm for PR based adaptive systems has been developed.
4. An efficient floorplanning tool targeting PR based designs has been developed.

1.7 Organization of This Report

Chapter 2 presents a detailed literature survey on partial reconfiguration covering different aspects such as architecture, design methodologies, tools and applications. Chapter 3 presents our method for automated partitioning for partial reconfiguration in adaptive systems design, Chapter 4 deals with the proposed floorplan method for partial reconfiguration using Columnar kernel tessellation, and in Chapter 5 we outline our future research direction.

1.8 Publications Resulting from Current Contributions

The work completed so far has been written up in a number of published and submitted papers

1. K. Vipin, S.A. Fahmy, *Efficient Region Allocation for Adaptive Partial Reconfiguration*, Proc. of IEEE International Conference on Field Programmable Technology (FPT), New Delhi, 2011.

2. K. Vipin, S.A. Fahmy, *Enabling High Level Design of Adaptive Systems with Partial Reconfiguration*, Proc. of IEEE International Conference on Field Programmable Technology (FPT), New Delhi, 2011.

3. K. Vipin, S.A. Fahmy, *Architecture-Aware Reconfiguration-Centric Floorplanning for Partial Reconfiguration*, Proc. of International Symposium on Applied Reconfigurable Computing, Hong Kong, 2012.

Chapter 2

Literature survey

In this chapter, we will review the development of partial and dynamic reconfiguration techniques over the years and their current state of the art. Although the terms partial reconfiguration and dynamic reconfiguration are frequently interchangeably used the literature, they can be different. By partial reconfiguration (PR), we mean a portion of the FPGA logic is modified while the remaining portions are not altered. This operation can be static or dynamic, meaning that the reconfiguration operation can occur while the FPGA logic is under reset state (static) or in running state (dynamic). It is not necessary that all dynamic reconfigurations are partial in nature. For example in context switching FPGAs, the whole configuration is changed during reconfiguration, but the operation is dynamic. In this chapter, we will analyse different aspects of PR including device architectures, design frameworks, PR development tools, optimisation strategies and applications.

2.1 Architecture

Conceptually all FPGA devices can be considered to be composed of two distinct layers: the configuration memory layer and the hardware logic layer [7] as shown in Fig. 2.1. FPGAs achieve their unique re-programmability as well as flexibility due this composition. The hardware logic layer contains the hardware resources of the FPGA. It includes lookup tables (LUTs), flip-flops, DSP blocks, memory blocks, transceivers,

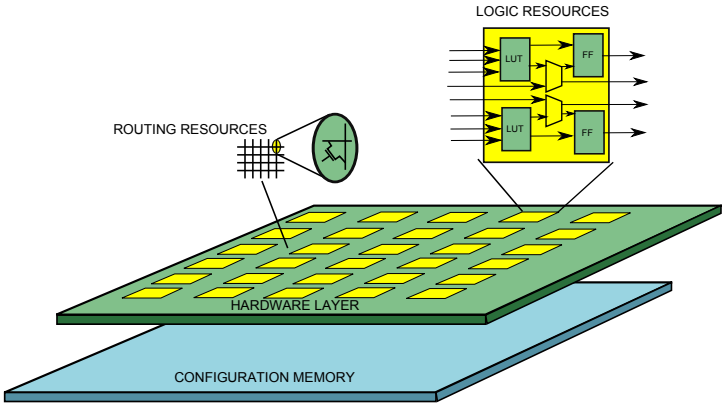


Figure 2.1: FPGA Architecture.

etc. Boolean functions can be implemented using LUTs, and flip-flops can be used for generating registers. This layer also contains the routing resources and the switch boxes to control them. The configuration memory layer stores the FPGA configuration information, usually called a bitstream. This bitstream contains information such as the values that need to be stored in the lookup tables, initial set and reset status of the flip-flops, initialisation values for the memories, standards of the input and output pins and the routing information for the programmable interconnects as well as the switch boxes. They function implemented by the hardware logic layer is determined by the values stored in the configuration memory. Configuration memory is usually SRAM based and hence volatile. Flash-based non-volatile configuration memory is present in some devices [8]. In order to change the circuitry implemented in the FPGA, the user has to simply modify the contents of the configuration memory by loading a new bitstream. The bitstream loading can be performed externally using interfaces such as JTAG, or SelectMap [9], or internally using specialised interfaces such as Internal Configuration Access Port (ICAP) [10].

Dynamic reconfiguration is a technique proposed for increasing the logic capabilities of FPGAs as well as to reduce the reconfiguration time. The limited resource availability in FPGAs was a major hurdle in implementing logic intensive applications. At the same time fetching configuration bitstreams from external memory causes long configuration time, which was unsuitable for several applications. For

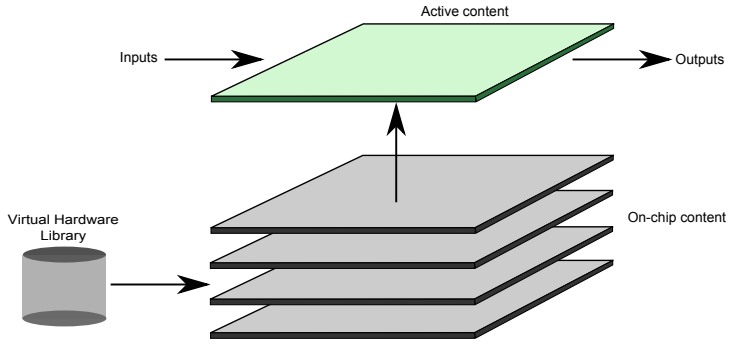


Figure 2.2: Multi-Context FPGA.

older generation FPGAs, only external configuration ports were available. The initial architectures proposed for dynamic reconfiguration were quite different from the presently existing architectures. Those were based on the concept that, since FPGA configuration bitstreams are stored in configuration memory, increasing the number of configuration memory planes will increase the FPGA logic density as well as reduce the switching time for changing from one configuration to another. This concept is quite similar to the context switching operation, and hence these FPGAs were generally called context switching FPGAs or Multi-Context FPGAs (MC-FPGAs) [11]. This concept is illustrated in Fig. 2.2.

The development of dynamically reconfigurable architecture can be dated back to 1995, when R. T. Ong from Xilinx Inc. filed a patent for an FPGA, which can store multiple configurations simultaneously [12]. In the initial design, there were two configuration memory arrays available in the FPGA. The two configuration memories were able to store different configuration data. During the first half of the user provided clock, the switches present at the output of the configuration memory cells select the configuration data stored in the first configuration memory array, and the logic and routing are configured accordingly. The results of the FPGA operation are stored in data latches. During the second half, the switches output the configuration data present in the second array and logic and routing are configured accordingly. The data present in the data latches at the end of the first cycle can be used during this second cycle. At the end of second cycle, FPGA outputs the results of its

function. This idea was further extended by Xilinx engineers in 1997 by proposing a time-multiplexed FPGA based on their XC4000E product family [13]. Here the FPGA can have one active configuration and up to eight inactive configurations. Each configuration memory cell is backed by eight bits of inactive storage in the configuration SRAM. This can be considered as eight configuration memory planes. Each configuration switching takes just 5ns and an additional 25ns for the signals to settle. There was an option to modify inactive configuration planes while the FPGA is running by loading off-chip configuration data. This brings a level of dynamic nature to the FPGA operation. A special mode of operation called the RAM mode allows user designs to read and write the configuration memory directly, which leads to the creation of self-modifying hardware. Special types of registers called micro registers were used for storing the CLB output while the FPGA changes configuration. Similar to Ong's proposal, data stored in these registers are available during next configuration and as an improvement, logic can access all values stored in micro registers from any configuration.

The main challenge associated with the adoption of MC-FPGA architecture is its high power consumption. Since a huge number of configuration bits need to be stored, the power requirement is in terms of tens of watts for a moderate design at 40MHz. This made the architecture unsuitable for many applications. A solution to this high power consumption and increase in area overhead due to running additional memory planes was suggested by Chong et al. with the introduction of reconfigurable context memory (RCM) [11]. RCM exploits the redundancy and regularity in configuration bits between different contexts. This architecture also depends upon a previous study which showed that during context switching, less than 3% of the configuration data is modified [14]. Additionally ferroelectric-based functional pass-gates are used as components of the RCM to achieve compactness and lower power. This design claims to reduce the FPGA area to 37% of conventional multi context FPGAs and consumes much lesser power.

The first practical context switching FPGA was developed by the researchers at Sanders, a Lockheed Martin company, on a 0.35μm platform at National Semiconductors [15]. This device is called context switching reconfigurable computer (CSRC),

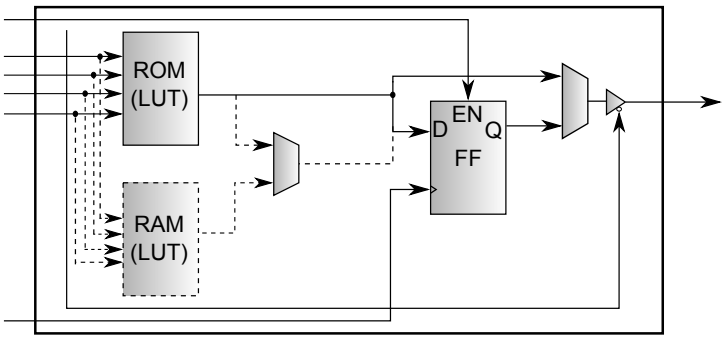


Figure 2.3: CSLC Architecture.

and can store up to four configurations concurrently. The device is arranged into 16-bit wide data pipes and each pipe is composed of context switching logic arrays (CSLAs). Each CSLA can process two 16-bit words and each CSLA is connected to two adjacent CSLAs which makes it possible to transfer data in both directions. The architecture uses three levels of routing for data to flow from any CSLA to any other CSLA. Each CSLA is composed of 16 context switching logic cells (CSLCs). Each CSLC contains a carry logic, a four input lookup table, a context switching flip-flop and a tri-state buffer. A separate context switching RAM is used for storage. Each configurable resource in this device along with the routing has four configuration bits, of which one bit is selected as active; thus achieving four configurable planes. This device is programmed serially using an external interface. A design framework for implementing applications on this device was developed by Puttegowda et al [16]. The context switching operation can be done under the control of a host processor or based on the request from the application running on the CSRC. A motion detection algorithm as well as enigma encryption algorithm are successfully demonstrated on this platform.

The first FPGA to support partial reconfiguration introduced by Xilinx was the XC6200 series [17]. This device supports true partial reconfiguration in a sense that only a portion of the FPGA can be reconfigured while the remaining portions continue to function. This device contains only a single configurable memory plane. The unique architecture of the FPGA makes it partially reconfigurable. The FPGA had a tiled architecture with each tile divided into a number of cells which contains

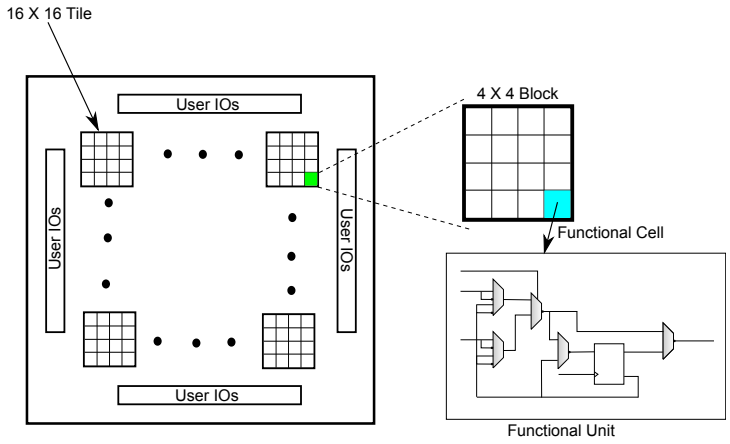


Figure 2.4: Xilinx XC6200 architecture.

functional cells. The functional cells are composed of a 2:1 MUX logic a flip-flop and routing resources. The MUX logic can implement any 2-input Boolean logic. Using a special interface, an external processor can access any specific cell in the FPGA. The processor is capable of modifying the configuration for any cell since the configuration SRAM is mapped to the processor address space. A relatively simpler PR method is possible for this FPGA compared to latest FPGAs due to its regular array architecture. Every cell and the routing to them are similar. Recent FPGAs have highly heterogeneous architecture and complex routing structures. Hence such a simple reconfiguration architecture and interface are impractical.

Partial reconfiguration started to become popular with the introduction of Virtex-II [18] and Virtex-II Pro [19] series of FPGAs from Xilinx. These FPGAs also contain built-in hard macros such as BlockRAMs and 18×18 embedded multipliers for efficient implementation of circuits. It was possible to load new data to the configuration memory while the remaining portions of the design continue to execute. A partial bitstream could be loaded externally using SelectMap or JTAG interfaces. In Virtex devices, Xilinx introduced a new configuration interface called the Internal Configuration Access Port (ICAP). Using this interface, it is possible to load the bitstream from within the FPGA design. A soft-processor or a custom state machine can fetch configuration information from external memory and write to the configuration memory.

In these devices, the configuration memory is organised as frames [20]. A frame is the smallest unit of configuration. One frame is one bit wide and extends the whole height of the device; hence the size of a frame is device dependent. All configuration operation should occur on a per frame basis. A configuration frame does not map to any single hardware resource, but it configures a narrow vertical slice of many physical resources. Configuration frames are grouped into six different configuration columns depending upon their hardware-mapping called IOB, IOI, CLB, GCLK, BlockRAM, and BlockRAM Interconnect. IOB columns are used for configuring the voltage standard for the I/Os on the left and right edges of the device. IOBs on the top and bottom edges of the device are configured by the CLB Columns with which they are vertically aligned. IOI columns configure the IOB registers, multiplexers, and 3-state buffers in the IOBs on the left and right edges of the device. IOBs on the top and bottom edges of the device are configured by the CLB columns with which they are vertically aligned. The CLB columns program the configurable logic blocks, routing, and most interconnect. BlockRAM configuration columns are used for programming the BlockRAM user memory space. The user design is not able to access BlockRAM while BlockRAM columns are being addressed by the configuration logic. BlockRAM Interconnect columns program all other BlockRAM and multiplier features, such as aspect ratios. The GCLK column configures most global clock resources, including clock buffers and Digital Clock Managers (DCMs).

For these devices there are several restrictions on the size and shape of the partial reconfiguration regions. The regions should extend the full height of the device and horizontally they should be in a four slice boundary. These restrictions often make the design inefficient in hardware utilisation, but the placement operation is relatively simpler. Tri-state buffers (TBUFs) have to be placed between the reconfigurable region and the static region in order to manage the connectivity between them.

Virtex-4 family of FPGAs [21] had a few architectural improvements compared to its predecessor, Virtex-II. The unreliable TBUFs are replaced by bus macros [22]. Bus macros are composed of LUTs as shown in Fig. 2.5. Since the locations of TBUFs were fixed, the connectivity between the modules was less flexible in Virtex-II family FPGAs. Since the new bus macros are LUT based, and LUTs are abundant in FPGA

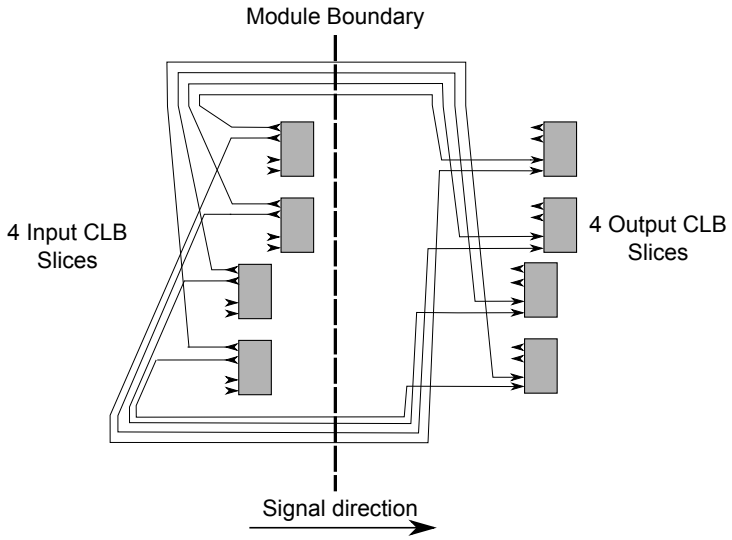


Figure 2.5: Bus Macro.

fabric, connectivity improved. Each bus macro is composed of 8 CLB slices. 4 slices are situated in the static region and 4 in the reconfigurable region. Separate types of macros are available for connecting modules from left to right and right to left. The size of frames is reduced in Virtex-4 [21]. Unlike Virtex-II, where frame size depends upon the device size, frame size is constant for all Virtex-4 devices. Each frame is 1 bit wide and 16 CLBs high and contains forty-one 32 bit words (1312 bits). The reconfigurable region does not need to span the full height of the device, but should be a multiple of 16 CLBs. The ICAP interface is upgraded to 32-bits wide from 8-bits wide as in the Virtex-II device. This considerably improved the reconfiguration speed.

In Virtex-5 FPGAs, the height of a frame spans an entire device row [23]. The number of rows in a device depends upon the size of the device. There can be 4 to 12 rows in a device. The FPGA is further divided into several blocks as shown in 2.6. Each block contains a type of FPGA component such as Configurable Logic Blocks (CLBs), DSP slices or Block RAMs. The device is composed of several tiles where a block and a row intersect, such as CLB tiles, DSP tiles BRAM tiles etc. One CLB tile contains 20 CLBs, similarly one DSP tile contains 8 DSP slices and one BRAM tile contains 4 Block RAMs. The number of frames present in each type of tile is

shown in Table 2.1.

Table 2.1: Tile types and number of frames in Virtex-5 FPGA.

Tile Type	Number of frames
CLB	36
DSP	28
BRAM	30

The number of bits in a frame is a constant, equal to 41 32-bit words or 1312 bits. From Table 2.1, it can be calculated that a CLB tile requires 47,232 bits for configuration, a DSP tile requires 36,736 bits and a BRAM tile requires 39,360 bits. Virtex-6 FPGAs follow the basic architecture of Vitex-5 FPGAs.

Altera has announced that their next generation Stratix-V FPGAs will be supporting dynamic reconfiguration at logic level [24]. Presently Altera's Cyclone-III and Stratix-IV FPGAs support dynamic reconfiguration in their transceivers such as PCIe, Gigabit Ethernet, PLLs etc, only. The user can dynamically modify several aspects such as transmission speed, clock rate etc. This helps in

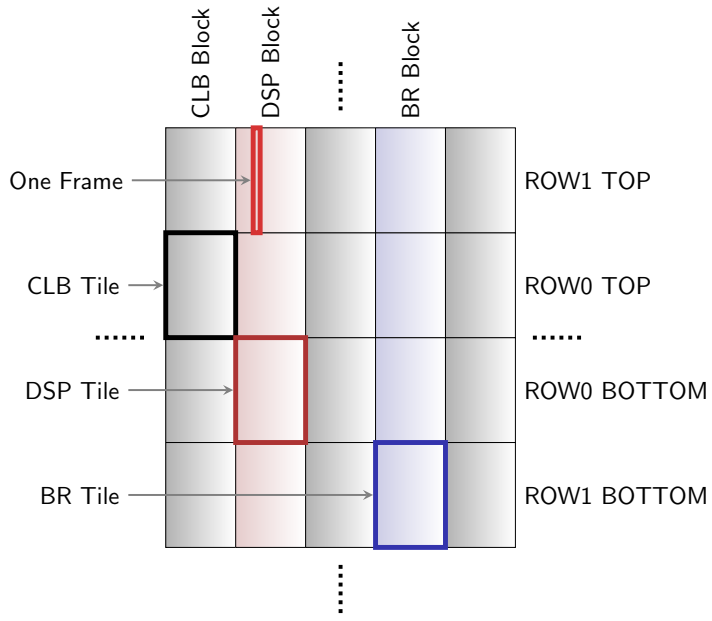


Figure 2.6: Virtex-5 FPGA architecture.

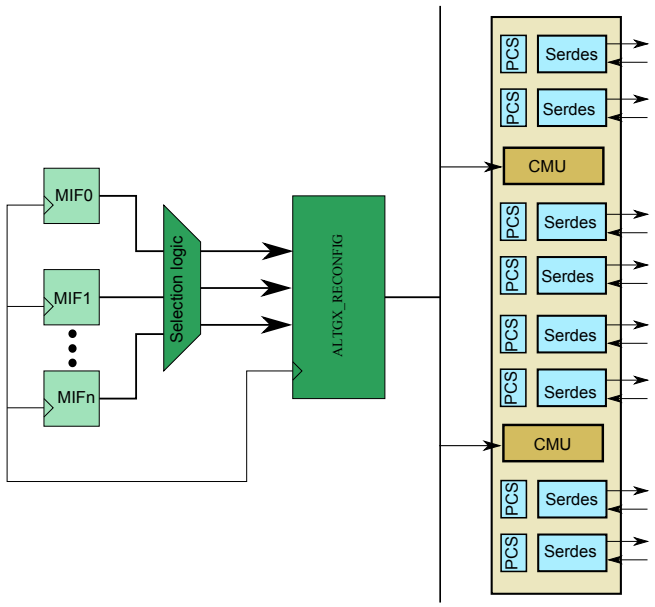


Figure 2.7: Altera PR Architecture.

- Universal front end (UFE) serial interface with flexibility to support multiple protocols (such as SONET, Fibre Channel and Gigabit Ethernet) using the same hardware
- Remote upgrades
- Continuous backplane signal integrity optimization to minimize bit-error ratio (BER)

A dynamic reconfiguration controller instance called the ALTGX_RECONFIG needs to be added to the design for dynamically reconfiguring the transceivers [25]. A single configuration controller can manage multiple transceiver blocks. The controller selects different transceivers by their logical channel address. A memory initialisation file, also known as .mif is used for reconfiguration purpose. A .mif file contains the information about the different settings of the transceiver module. Each word in .mif is 16 bits wide. The dynamic reconfiguration controller writes information from the .mif file to the transceiver module as shown in Fig. 2.7. The logic PR flow for Stratix-V devices is not yet formalised.

Lattice semiconductors provide ORCA series FPGAs as their PR supporting device [26]. ORCA is a coarse grained FPGA with programmable logic cells (PLCs), programmable I/Os, and embedded RAMs (EBRs). The PLCs are arranged in an array of rows and columns. Each PLC consists of a programmable functional unit (PFU), system level interconnect (SLIC), and routing resources. Each PFU contains eight 4-input LUTs, eight latches/FFs, and one additional flip-flop that may be used independently or with arithmetic functions. The design is synthesised and implemented using standard Lattice-ISP software tool. PR is done by setting a bitstream option in the previous configuration sequence that tells the FPGA to not reset all of the configuration RAM during a reconfiguration. Then only the configuration frames that are to be modified need to be rewritten. Other bitstream options are also available that allow one portion of the FPGA to remain operational while a partial reconfiguration is being done.

AT40K series FPGAs from Atmel supports both partial as well as dynamic reconfiguration [27]. The AT40K architecture is symmetrical array of identical cells and is fine grained. These FPGAs offer distributed 10 ns SRAM capability, where RAM can be used without losing logic resources. The AT40K FPGAs have 8-sided core cells with direct horizontal, vertical, and diagonal cell-to-cell connection that implements ultra fast array multipliers without using busing resources. Atmel FPGAs can be designed using industry-standard EDA tools such as Mentors Precision Synthesis for VHDL and Verilog on a PC platform. Atmel's Integrated Development System (IDS) software is used for place and route operations. These FPGAs are ideal for building adaptive filters, variable coefficient multipliers, etc. but unsuitable for logic intensive applications due to limited logic capacity.

3D programmable logic devices are commercially available from Tabula Inc.[28]. These devices use a technique known as Spacetime technology. In a Spcaetime device, logic, memory, and interconnect resources are dynamically reconfigured up to eight times in each user cycle. This is similar to the context switching FPGA concept. The Spacetime compiler automatically maps, places, and routes a user design into the 3D device using standard VHDL/Verilog inputs and flows. To enable rapid reconfiguration, configuration data are stored locally to the resources they control and

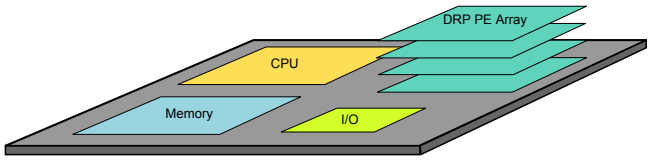


Figure 2.8: MuCCRA-Cube Architecture.

this local configuration memory is made to look like a stack. As each configuration is read from the top of the stack, the next configuration rises to the top. The current configuration goes to the bottom of the stack. This process repeats continuously.

Other than commercial FPGAs, there are number of reconfigurable architectures proposed by the research community [29, 30, 31, 32]. MuCCRA-Cube is such an architecture proposed by Saito et al [31]. It is a scalable three dimensional dynamically reconfigurable processor. By stalking multiple dies connected, the number of processing arrays can be increased. The basic architecture of this device is shown in Fig. 2.8. A mother chip provides a CPU, a memory module, a reconfigurable core, and standard I/Os. The number of reconfigurable cores can be increased by stacking corresponding daughter dies. Each PE array structure in this device is composed of a 4×4 24-bit PE and four 24-bit distributed memory modules (MEMs). Data can be transferred along multiple routing channels in any direction, horizontally or vertically by using switching elements. Each MEM is 24-bit $\times$ 256-word dual port SRAM module and hence the PE array I/O can be performed independently. Different dies are stacked by means of inductive coupling.

DART is a dynamically reconfigurable architecture especially targeting future mobile communications [30]. Since future communication systems should be able to handle several tasks concurrently, DART has been broken up into clusters, as shown in Fig.2.9. Each cluster of DART integrates two kind of processing primitives, Reconfigurable DataPath (DPR) and an FPGA core. The DPRs are reconfigurable at the functional level in order to optimise the interconnections between arithmetic operators. FPGA core is reconfigurable at the bit level in order to efficiently support logic processing. Each cluster of DART integrates one FPGA core and six DPRs.

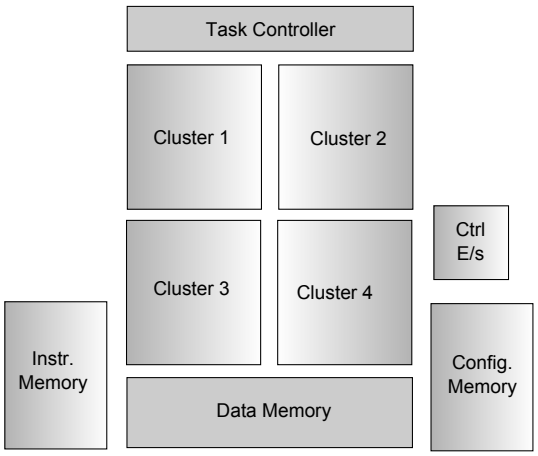


Figure 2.9: DART System Architecture.

The arithmetic processing primitives in DART are the DPRs. Each DPR is organised around functional resources and memories, interconnected according to a very powerful communication network. DPR has 4 functional units followed by a register. The dynamic reconfiguration is carried out by the cluster controller. With six DPRs in each cluster, the configuration data size is 772 bits.

2.2 Design Methodologies and Tools

In this section we review the design methodologies for PR systems and the supporting tools developed for making the system implementation simpler. We will review both industry design methods as well as the methods proposed by the research community.

2.2.1 Xilinx PR Flow

Presently the only vendor-supported PR flow is available from Xilinx Inc. Xilinx supports PR through their software package called PlanAhead. A hierarchical module based design flow has to be followed for achieving the required system performance. Xilinx uses specific terms for denoting different attributes of PR. Each PR design is composed of number of *modules*. A module can be considered as a functional unit, for example a filter. A module can have a number of *modes*. Modes are different

implementations of the same module. For example a filter can have two modes, a high pass filter and a low pass filter. Usually modes of the same module will have similar interfaces. All modules are described using some hardware description language or can be pre-synthesised netlists.

The whole design is composed of two sections, a *static region* and *reconfigurable regions*. A static region is the portion of the design, which does not change its functionality during the FPGA operation. Usually the static region contains a processor running the reconfiguration management software, internal configuration interface and memory interface modules. Dynamic regions implement the reconfigurable modules. Their functionality can change during runtime. A single reconfigurable region can implement a number of modes in a time multiplexed fashion. The first design step is the generation of different module modes and the static region. They can be hand coded or Standard IPs can be used. After that they are synthesised using the XST tool to generate the netlists. Then the floorplanning has to be done. The current tool flow requires the designer to perform this operation manually. A region of the FPGA has to be selected, which contains all the required FPGA resources for the implementation of modes assigned to it. The regions needs to be rectangular in shape and should be aligned to tile boundaries. Regions which are not aligned to tile boundaries can also be implemented, but this requires additional circuitry. This occurs since configuration occurs on a per tile basis. If a tile is shared by two reconfigurable modules, reconfiguring one region may lead to altering the portion of the tile belonging to the other region. This may cause issues. In order to prevent this, a read-modify-write back circuitry has to be implemented. This circuit reads the contents of the tile, modifies the portions belonging to the reconfiguring region and then writes back. This operation not only requires additional circuitry for performing the operation, but increases the reconfiguration time overhead. Hence generally tiles are not shared between reconfigurable regions.

Reconfigurable regions are decided based on the modes assigned to each region. This partitioning of modes to different regions has to be performed manually. Each region should be as large as the largest mode assigned to it. The set of valid combinations of modes present in the design are called *configurations*. The designer assigns

each region with modes so that valid configurations are generated. The tool generates a full reconfiguration bitstream as well as partial bitstreams for each reconfigurable region for each configuration.

Another flow for PR was available from Xilinx known as difference-based partial reconfiguration [33]. This flow was used for making small changes in the design. The minor changes are added by editing the design using *FPGAEditor* GUI software. Then Xilinx tools are used to generate the partial bitstream, which contains only the difference made from the base design. This flow is no longer recommended for PR designs.

2.2.2 Proposed High Level Design Methods

Researchers have proposed number of tools and methodologies for generating PR based systems. There are specific methods for modelling and optimising PR systems [34, 35]. Due to the lack of public availability and lack of supported architectures, many of these tools have not found wide acceptance. Many of the tools consider customised FPGA architectures, which are practically not available.

A tool called SynDex [36] can be used for automatic generation of PR systems [37]. This tool works based on AAA methodology, which corresponds to Application, Architecture and Adequation. It aims at finding the best matching between an algorithm and an architecture while satisfying timing constraints. The application algorithm is represented by a data flow graph. Architecture is also modelled by a graph with the vertices being the operators. Adequation consists of performing the mapping and scheduling of operations and data transfers onto the operators and communicating media. Once mapping and scheduling of the algorithm are performed, macro-code is automatically generated and translated to VHDL code for both static and dynamic parts. The runtime reconfiguration manager uses prefetching to minimise reconfiguration latency.

A set of CAD tools for PR has been developed by Robertson and Irvine [38]. These tools include options for design specification, simulation (functional and timing), synthesis, placement and routing, partial configuration generation and control of

partially reconfigurable designs. A tool called DCSim is used for simulation, DCSTech for placement, and DCSBitstream for bitstream generation along with Xilinx tools. Similarly GePaRD is a design flow used for high level generation flow for PR designs [39]. The GePaRD flow enhances Xilinx PR flow by a high-level synthesis framework. The flow uses a high-level specification of the PR system as input and generates both a system model for simulation and a physically-aware architecture description as input for the implementation on the target device using Xilinx PR design flow. The design flow includes template abstraction, high-level synthesis, and temporal modularization. An object-oriented frame work for PR design and implementation is presented by Abel [40]. This framework consists of a software-to-hardware compiler, a NoC also reliable for data buffering, a merger, and an adaptive scheduler and a Java emulator. These tools also interact with Xilinx PR tools for generating configurations.

A generic software framework for PR based adaptive systems is presented in [41]. In this method, an adaptive system is considered as having two planes. The data plane implements the processing of data, such as the signal processing in a radio. The system designer uses a high level tool to describe this based on a library of IP cores. The control plane implements the management and control functionalities. This plane is implemented in software due to its flexibility. Adaptations are managed by the control plane by determining the appropriate type of configuration and loading the corresponding bitstreams or configuring internal register values.

Another layer based architecture is presented by Tan and DeMara in [42]. The hierarchical framework considers mainly three aspects, autonomous operation, task-level modularity and runtime scenario support. The different layers are logic layer, translation layer and the reconfiguration layer. The logic layer supports general user-level applications, carrying out hardware-independent logic control on the tasks running on the FPGA. The translation layer translates logic descriptions for the tasks into specific physical details as reconfiguration data (bitstreams). The reconfiguration layer includes hardware platform and the low-level communication APIs.

2.2.3 Partial Reconfiguration Support Tools

In this section some of the tools and algorithms developed for supporting PR are reviewed.

Partitioning

Partitioning involves dividing the design into a number of functional units and grouping them for efficient implementation of the system. Conger et al. [43], clearly identify the challenges associated with PR designs and point out the need of more advanced tools for guiding PR designers. They present a design flow for two different kinds of PR systems, one in which all the information needed for system implementation is available beforehand and the other in which the application of the system can vary after deployment. A brief introduction about region partitioning is also provided although no specific algorithm is provided.

A recent work that explores partitioning and floorplanning of PR designs is presented by Montone et al. [1]. The authors describe a simulated annealing based algorithm for finding out the module allocation to regions based on the minimisation of area requirement variance at different time instances. This work considers the latest architecture of FPGAs as well as PR requirements. It is difficult to extend this work for adaptive systems because the algorithm presented uses a scheduled task graph. Moreover the impact of reconfiguration time is not accounted in their method. [44] and [45] describe floorplanning methods for partial reconfiguration once manual partitioning has been performed by the designer.

Adaptive systems differ from task based systems over an important aspect. In task based systems, either the tasks which will be executed in the system and their sequence are known in advance or it is impossible to predict the sequence of tasks that will be executed during runtime. In the earlier case, it is possible to apply efficient partitioning schemes during design time itself considering the order of execution. For adaptive systems, the tasks which will be executed on the system are known beforehand, but it is impossible to predict the order of execution. The order depends upon external factors. Hence an efficient offline partitioning has to be performed

considering the dynamic nature of the system. Since the partitioning is done offline, the quality of the partitioning, not the speed of the algorithm, is critical.

Most of the existing works we have found do not perform reconfiguration aware partitioning and do not consider latest FPGA architectures. Hence, the partitioning is not optimised for the dynamic behaviour of a partially reconfigurable system. In our work [46], we introduced a method for PR partitioning considering the dynamic nature of adaptive systems and architectures of current FPGAs. The primary object of this work was to minimise the total resource consumption as well as the average reconfiguration time. The optimal solution was determined based on linear programming technique. This paper is an improvement of our previous work by removing several design constraints. We discuss this work in detail in Chapter 3.

Floorplanning

Several approaches to FPGA floorplanning have been published, although work related to floorplanning for PR is less abundant. Traditionally, FPGA floorplanning is considered as a fixed-outline floorplanning problem, as introduced in [47] and further extended in [48]. The authors present a resource-aware fixed-outline simulated-annealing and constrained floorplanning technique. Their formulation can be applied to heterogeneous FPGAs but the resulting floorplan may contain irregular shapes, which are not allowed in current PR designs. Another interesting study is presented in [49], which presents an algorithm called "Less Flexible First (LFF)". In order to perform placement, the authors define the flexibility of the placement space as well as the modules to be placed. A cost function is derived in terms of flexibility and a greedy algorithm is used to place modules. The generated floorplan will have only rectangular shapes, but the approach only addresses older-generation FPGAs and is unsuitable for recent families due to their heterogeneous resources.

Findings in [50] are based on slicing trees. Using this method it can be ensured that the floorplan contains only rectangular shapes. Here, the authors assume that the entire FPGA fabric is composed of a repeating basic tile, which contains all types of FPGA resources including Configurable Logic Blocks (CLBs), Block RAMs and DSP slices. Although this assumption is valid for older-generation FPGAs, such as

the Xilinx Spartan-3, more recent FPGAs such as the Xilinx Virtex-5 family, do not have such a repeated tile architecture.

Yuh et al. have published two methods for performing floorplanning for PR. One method is based on using a T-tree formulation [51] and the other is based on a 3D-sub-Transitive Closure Graph (3D-subTCG) [52]. Using T-trees, each reconfigurable operation is represented as a 3D-box, with its width and depth representing the physical dimensions and its height being the execution time required for the operation. Here the reconfiguration operations are at task level rather than functional level and the authors consider older-generation Virtex FPGAs, which require columnar reconfiguration.

Montone et al. present a reconfiguration-aware "floorplacer" in [1]. They consider the latest architecture of Virtex-5 FPGAs. Initially the design is divided into reconfiguration areas based on the minimisation of temporal variance of the resource requirement. Then a floorplacer tries to minimise the area slack using simulated-annealing. In [53] a floorplanning method based on sequence pairs is presented. In this work, authors have shown how sequence pairs can be used to represent multiple designs together. Here, designs are the circuitry present in the FPGA at different instances. An objective function tries to maximise the common areas between designs and simulated-annealing is used for optimisation. Although simulated-annealing-based floorplanners have been developed, for soft modules, which are common in PR designs, the results are not satisfactory [54].

All existing work we have found focuses on the static properties of a particular placement. Hence, the placement is not optimised for the dynamic behaviour of a partially reconfigurable system. Other work relies on fixed task-graphs and hence only optimises for a fixed sequence of configurations.

Runtime Placement

Placement operation involves finding a suitable location for a reconfigurable module during runtime. Unlike static FPGA designs, in PR based systems, modules have to be placed as well as removed during runtime. Bazarghan et al. considered this placement as an on-line bin-packing problem [55]. Later, Lu et al. introduced an

algorithm for online task placement [56].

Another method for online placement and removal of modules is presented in [57] for Virtex-II Pro FPGAs. This work focuses on a method that enables arbitrary removal and placement of hardware implementations and performs the necessary routing to disconnect and connect them to other implementations present in the device. Before assigning a new module to a region, the interface of the previous module is unrouted to prevent any damage to other modules. This work considers designs only containing CLBs. Sandors et al. proposed a method to improve the placability of modules with the help of defragmentation [58]. Continuous addition and removal of modules to the FPGA causes the free space to become fragmented, which prevents the allocation of new modules. A suitable defragmentation algorithm maximises the continuous free-space available for module placement. In [59] defragmentation is used for relocating modules.

In [60] a method is proposed for increasing the placability of reconfigurable modules. The authors consider partially reconfigurable regions consisting of reconfigurable tiles. This method can handle FPGAs containing heterogeneous resources such as BRAMs, DSP slices etc. The algorithm defines the set of feasible positions for PR modules and optimises the regions in such a way as to minimise the degree of overlap with other regions. An overlap graph is used for this purpose. Another method for improving the placability is described in [61]. This algorithm is targeted for Virtex-4 FPGAs. The technique utilises a compatible subset of resources in non-identical regions. This makes it possible to place modules at non-identical regions.

Several tools are developed for online module placement targeting different FPGAs. One of the widely used tools is PARBIT (PARtial BItfile Transformer) targeting Virtex-E FPGAs. Modules are relocated by manipulating the contents of the partial bitstreams. To generate a new placement, PARBIT reads the configuration frames from the original bitstream and copies to the partial bitstream only the configuration bits related to the new area. It then generates new values to the configuration address registers, according to the partial reconfigurable area. Similar to PARBIT, REPLICA (RElocation Per onLIne Configuration Alteration) [62] is another filter tool targeting Virtex-E FPGAs. Specific system architecture is needed for using these tools, in which

modules are placed along horizontal communication architecture. This is compatible with Virtex reconfigurable region requirements. The filter is implemented in the FPGA itself and performs the address manipulations for relocation during run-time. Replica2Pro [63] is an advanced version of this filter, which supports Virtex-II and Virtex-Pro FPGAs. This filter can support relocation of BRAMs as well as multiplier blocks.

The major disadvantage of online place-and-route tools are their lack of portability. Due to architectural variations, new tools have to be developed even for FPGAs belonging to the same family. The lower level configuration frame details available from Xilinx has considerably limited after the Virtex-5 FPGAs. Even for FPGAs before Virtex-5, researchers use numerous trial and error methods for finding out the effect of different configuration bits. Most of the tools support very few FPGAs belonging to the same family due to long man hours required for decoding the function of different configuration frames. Support tools such as JBits [64] are no longer endorsed by Xilinx. We feel that, it will be more productive and efficient to use the vendor-provided tools for performing the lower-level architecture dependent operations such as placement, since the real challenge is at the higher levels of design, and linking with supported tools means the results of our work are more likely to be compatible with future devices.

Communication

Communication between reconfigurable modules as well as between the reconfigurable modules and the static region are essential in the efficient operation of the system. Ahmadinia et al. proposed the RMBoC (reconfigurable multiple bus on-chip) architecture [65]. Communication links connect switches in a linear array. Architectural parameters include number and width of communication links. Switches have only one input and one output port for module connections. Switches use a centralized FIFO and arbiter to receive all connection requests. Switches dynamically establish dedicated communication paths between modules, although, the architecture does not leverage flow control, which is problematic when a module exhausts buffer memory.

A run-time communication synthesis technique is introduced in [66]. In this

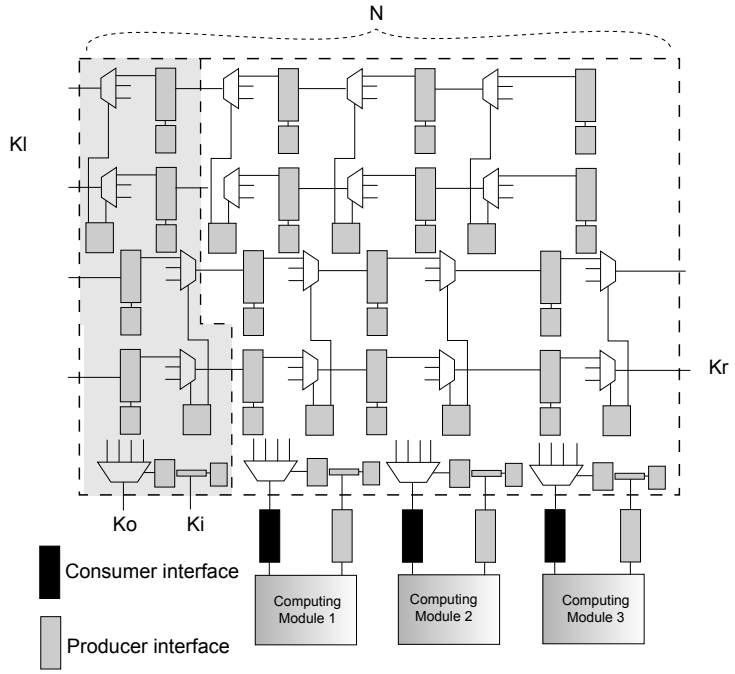


Figure 2.10: SCORES Communication Architecture.

method, partial bitstreams of IP blocks are stored in a dynamic module library. Before compilation, blocks are encased in wrapper structures whose main function is to provide routing anchor points for block ports. Multiplexers allow run-time selection among same-side and opposite-side connections to module ports, and pass-through connections for signals unrelated to the module.

ReCoBus [67, 68] is another bus architecture supporting PR. This architecture supports point-to-point communication links between the static part of a system and reconfigurable modules. This technique is called I/O bars. A ReCoBus is able to implement all established bus protocols including AMBA, CORECONNECT, AVALON, or Wishbone. The bus can contain pipeline registers in order to enhance the throughput. The physical specification of a ReCoBus can be reused by multiple designs, and such specifications are independent of target FPGA, hence the same specification may be used to build systems for different FPGA families.

Fig. 2.10 shows the top-level design of the SCORES communication architecture [69]. SCORES is composed of a linear array of switches. Each switch has a unique X

coordinate indicating its horizontal position inside the linear array. Switches communicate with neighbouring switches (Kl and Kr) and computing module interfaces (Ki and Ko) through bidirectional communication links between their input and output ports. Computing modules attach to switches through two types of module interfaces. Consumer Interfaces connect a computing modules input port to a switchs local output port (Ko). Producer Interfaces connect a computing modules output port to a switchs local input port (Ki). Dynamically established Dedicated Streaming Routes (DSRs) enable data transmission between two computing modules. These dedicated routes provide high throughput and low latency data transmission. A computing module can be both a producer and consumer simultaneously.

2.3 Optimising Partial Reconfiguration

Although partial reconfiguration has several advantages, it brings additional overheads to the system. The primary overhead associated with PR is reconfiguration time. Reconfiguration time is the time required by the system to switch from one operating mode to another. It usually involves reconfiguring portions of the FPGA and this requires time. Excessive time for the reconfiguration operation can affect the system efficiency. Hence a number of techniques are proposed for reducing the reconfiguration overhead.

One of the straightforward methods to reduce reconfiguration time is to increase the speed of the reconfiguration operation itself. Reconfiguration is usually performed with the help of dedicated hardware such as Internal Configuration Access Port (ICAP). By increasing the speed of the ICAP interface, reconfiguration time can be reduced. Previous studies show that the effectiveness of PR increases as the speed of ICAP increases [70]. A fully streaming DMA engine connected to the ICAP interface increases the reconfiguration speed up to its maximum limit. An area-efficient ICAP controller directly connected to the Processor Local Bus (PLB) and using DMA is shown to achieve a speedup of up to 20 times in Virtex-II Pro FPGAs [71] compared to non-DMA based reconfiguration. Another method to improve the ICAP performance is presented in [72] by using an enhanced ICAP hard macro. Here the Xilinx

ICAP primitive is extended with custom logic. This makes it possible to overclock the ICAP primitive in a controlled manner and achieve a maximum throughput speed of 17056 Mbits/s, which is five times faster than the conventional ICAP throughput.

Bitstream compression is another widely used technique for reducing configuration time [73, 74, 75, 76, 77, 78]. In [73], researchers exploit redundancies both within a configuration bitstream as well as bitstreams of different configurations. Their analysis shows that frames configuring CLBs have a high degree of regularity among themselves. Huffman encoding is used to compress the bitstreams. [74] and [75] present an algorithm to compress bitstreams for Xilinx XC6200 FPGA. The technique results in an overall reduction of configuration time by a factor of 4. The algorithm generates a new configuration file from the original, with fewer configuration writes by using the Wildcard Registers present in the FPGAs. [76] and [77] present algorithms for bitstream compression for Virtex FPGAs using different compression techniques such as Huffman coding, Arithmetic coding, LZ coding, etc.

Ganesan and Vemuri published a method for reducing reconfiguration time using integrated temporal partitioning and partial reconfiguration [79]. This paper describes using a partially reconfigurable processor having two reconfigurable regions for execution speed up. Configuration for the subsequent instruction is loaded to a reconfigurable region, while the current instruction executes in the other region. A similar work is presented in [80], in which a prefetching scheduling is also generated based on control flow graphs. Unfortunately for adaptive systems, direct scheduling based on task graphs is not possible due to their dynamic nature. It is difficult to predict the order in which the system will vary its configuration. In [81], authors present a method for minimising the reconfiguration latency based on communication graphs. The algorithm presented tries to group modules with highest communication cost in the same reconfigurable region. Here the weakness is that the number reconfigurable regions available in the device has to be provided by the designer. Determining the number of regions is not straightforward, and current devices and tools do not provide any support in this respect.

Configuration caching is another method suggested for reducing re-configuration time. Using the technique described in [82], overhead is reduced by a factor of 5. Here

the area of the FPGA is viewed as a cache. Configuration management techniques are used to determine when configurations should be loaded and unloaded to minimise the overall reconfiguration time. For PR platforms executing task level configurations, optimal scheduling algorithms are also developed for minimising reconfiguration time [83, 84].

2.4 Applications of Partial Reconfiguration

A number of applications have been developed which exploit the unique characteristics of PR. Several applications which are currently implemented using traditional FPGA design methodologies can improve their efficiency by adopting PR design methods. PR based designs developed ranges from image processing and communication to aerospace and space applications.

One immediate application of PR is in software defined radio (SDR) development [4]. The dynamic nature of PR is essential in achieving the flexibility requirements of SDR. Frameworks for developing SDR based on PR are already proposed [3] [85] [86]. Cognitive radios are motivated by the fact that the current radio spectrum allocation is not efficiently utilised and the efficiency can be improved by allowing secondary users to use the spectrum while the primary users are not using it. This requires the system to be quite adaptable, changing a number of processing aspects such as modulation, encoding, decoding etc. at runtime. It is proposed that cognitive radios can be implemented using PR by considering the system composed of two parts [85]. The Processor Subsystem (PS), which integrates the hardware modules required to run a standard Linux operating system and a Customisable Processing Subsystem (CPS), which is used of implementing components with high computational requirements. Processing elements for cognitive radios such as reconfigurable FIR filter are already developed [86].

Another possible application field for PR is in audio and video applications. Processing cores such as MP3 decoder [87], JPEG encoder [88] etc. are already implemented using PR. In [89], a PR based scalable H.264/AVC deblocking filter architecture is described. The filter adapts to different users' requirements intelligently.

A real time video processing system using PR is described in [90]. Different types of image processing filters such as mean and median filters are implemented in the same reconfigurable region so that the resource requirement and power consumption are reduced.

PR promises extensive applications in space and avionics. One of the important hurdles in using FPGAs in space applications is the Single Event Upset (SEU) effects [91]. An SEU is a change of state caused by ions or electro-magnetic radiation striking a sensitive node in a micro-electronic device such as semiconductor memory. SRAM based FPGAs such as Xilinx and Altera FPGAs are highly vulnerable to SEU, which leads to corruption in the configuration memory and may cause serious system damage. PR has been proposed as a method for mitigating SEU effects on SRAM based FPGAs [92] [93] [94] [95]. In [92] authors propose a method to partition the FPGA into number of regions in order to isolate the SEU errors based on duplication with comparison. Once an error is detected, that region is reconfigured. Design is partitioned into different regions by considering different cost factors such as size of the subsystems, size of the data width to be compared and the target FPGA architecture. Another simple method to overcome SEU using PR is by configuration scrubbing [94]. In the simplest form, the configuration data is stored in a radiation hardened memory and a configuration controller configures potions of the FPGA using this memory periodically. This is called blind scrubbing. In an advanced method, the configuration controller reads data from the FPGA and detects the presence of an error and writes back configuration data only if an error is present. Advanced SEU mitigation using both PR as well as traditional TMR methods are also suggested [96].

PR finds applications in networking also. Within space applications, [97] describes the implementation of System-on-Chip Wire (SOCWire) architecture on a partially reconfigurable Virtex-4 FPGA. SOCWire is well established in space community with supports such as Link initialization, Credit based flow control, Detection of Link Errors, Link Error Recovery, Hot-plug ability etc. This dynamic characteristic makes it an ideal candidate for PR based implementation. A packet processing system called Field Programmable Port Extender (FPX) is developed based on PR [98]. FPX

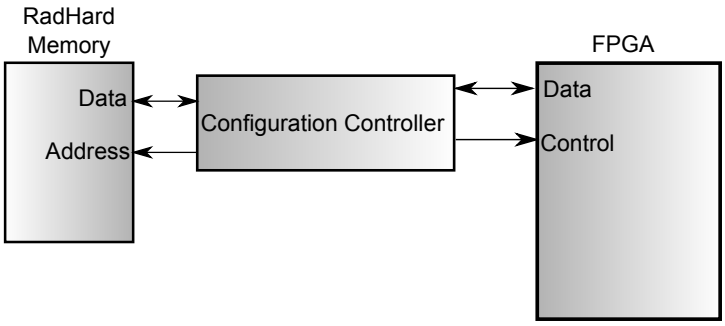


Figure 2.11: Configuration scrubbing.

contains logic to transmit and receive packets over a network and dynamically reprogram hardware modules and route individual traffic flows. The reconfigurable virtual network presented in [99] combines several partially-reconfigurable hardware virtual routers with software virtual routers. Hardware virtual routers are configured using dynamic reconfiguration. Functions such as header verification, checksum verification, IP lookup, ARP lookup, and time to live (TTL) updates etc. are implemented in PR regions. All these functions can be implemented in a single region since they operate sequentially on a packet. The forwarding table for the virtual router is also stored in PR regions and this can be updated via a PCI bus using a host software. This study shows that network implementation based on PR gives much higher throughput and forwarding performance.

Researchers have shown the potential of PR in automobile applications, especially in driver assistance systems [100]. Video standards for driver assistance systems are not standardised and are subjected to constant upgrade. In automobiles, frequent upgrade of hardware components is not feasible. PR makes an ideal choice for this upgrade. In [2], authors present a method for video based driver assistance using PR. The system uses a Power PC processor present inside the FPGA for control and management purposes. The different image processing functional units are implemented as co-processors. The example design implements three different addressEngines (AEs), which are used for selecting the pixels for processing. The addressing can be inter, intra and segment addressing.

PR finds applications in high energy physics experiments also. For example PR

is used in the Compressed Baryonic Matter (CBM) experiment conducted at the Facility for Antiproton and Ion Research (FAIR) in Darmstadt, Germany [101]. This experiment used an Active Buffer Board (ABB) for receiving, buffering and forwarding hit data. In a high energy physics experiment, the surrounding conditions can change. Thus it was required that the ABB functionality had to change after the installation of ABB into the system. For this purpose PR was used.

Another interesting application is the development of dynamic instruction set computer (DISC) [102]. This processor supports demand-driven modification of its instruction set. Each instruction in the instruction set is implemented as an independent circuit module. The individual instruction modules are paged onto the hardware in a demand-driven manner as dictated by the application programme. Hardware limitations are eliminated by replacing unused instruction modules with usable instructions at run-time.

In [103], Steiner has proposed using PR for creating autonomous computing systems. This model proposes the implementation of placing and routing logic on the FPGA fabric itself. The main challenge here is the logic overhead due to these implementation tools and the level of intelligence required building into the system.

PR is successfully used for pattern recognition and computer vision. For example [104] presents an on-line evolvable pattern recognition system. In this system, the classification module is dynamically evolved using PR. In [105] AdaBoost algorithm for human detection is implemented on a Virtex-4 FPGA based on PR. Two computationally intensive tasks, integral image computation and feature extraction/decision, are implemented in a single PR region. The output of the first operation is used as the input for the second operation. For this design, CLB usage for reconfigurable implementation is much less compared to the complete static implementation. Other applications of PR include image processing [106], communication [107] and algorithm implementations [108] [109].

2.5 Summary

PR has evolved significantly over the past years. It finds extensive applications in different fields including scientific experiments and aerospace. The heterogeneity of modern FPGAs makes it harder to follow general FPGA design approaches in PR based systems design. Several of the research results are not feasible in real systems, mainly due to the differences in the assumptions made from practical scenario. Most of the existing tools are target architecture dependent. This means, when architectures evolve, the tools are no longer usable. Hence it is better to use vendor-supported tool flow for adaptive systems design by increasing the automation and level of abstraction for efficient implementation as well as portability. The unpredictable nature of adaptive systems is a real challenge in developing a design framework for them. The framework should be aware how the system will adapt depending upon the environmental conditions and optimise the design globally considering these different operating circumstances. The supporting tools should be able to make PR based adaptive systems design easy for non FPGA experts.

Chapter 3

Partitioning for Partial Reconfiguration

3.1 Introduction

Although runtime partial reconfiguration has advantages over full reconfiguration, it presents a greater challenge to the designer, and current tools impose a number of limitations. The designer must decide on how to partition the FPGA between the *static*, always-on, parts and reconfigurable regions, requiring detailed knowledge of the FPGA architecture. The designer must also determine which modules should be assigned to which regions and hence the granularity of reconfiguration. Some work has been done to mitigate these limitations, but existing approaches are only suitable for use by FPGA experts. Hence the system designer and the module designer are typically one and the same, with extensive hardware experience. In an ideal world, application experts should be able to leverage this capability given the right tools.

For systems incorporating partial reconfiguration, design decisions can impact the cost of using PR. One of the benefits of partial reconfiguration is the ability to time-multiplex functionality, resulting in decreased area requirements compared to an implementation that does not use PR. However, the number of regions used, and the allocation of reconfigurable modules to regions can impact the requirements considerably. Hence for greater savings, this allocation should be done intelligently.

The primary cost of partial reconfiguration, when considering adaptive systems, is reconfiguration time. In a circuit in which all functionality is present on chip, selecting between modes will typically take just a few clock cycles. When using PR, the system must pause, and the PR region must be reconfigured, which takes time. Smaller regions result in shorter reconfiguration times, but can impact area efficiency. Hence we try to optimise these two factors together to arrive at an optimal allocation.

Most existing research has pursued a bottom-up, architecture-dependent approach to simplifying and generalizing partial reconfiguration. The challenge is that as vendors evolve their architectures the assumptions made are broken and techniques must be reformed. Hence, methods that apply to one generation of FPGA, allowing modules to be placed in a regular grid pattern, can be broken when the next generation of devices alters the pattern. While bottom-up architecture-based research such as online placement have proven interesting, recent devices with more heterogeneous blocks and more complex non-uniform routing make these approaches harder to implement and keep up to date [110]. Hence we feel it is more effective to develop tools that integrate with vendor-supported tool flows to address the challenges of partial reconfiguration, focussing more on top-down simplification of this unique feature of FPGAs for use by those who are not architecture experts. Furthermore, as vendor tools increase in sophistication, the effectiveness of these tools can also increase.

In this chapter we present techniques for automatically determining the optimal reconfiguration scheme for a given adaptive application. Based on a representation of the application, the tools propose the best arrangement of reconfigurable regions and how modules should be assigned to those regions. We are most interested in adaptive applications where reconfiguration occurs at the module level, such as a change in coding scheme in a radio system. This sort of reconfiguration occurs on an as-needed basis, and is at the functional, rather than task, level. This means we cannot predict the order and frequency of reconfiguration, so must optimise globally to reduce reconfiguration time and minimise area. The aim is that an adaptive systems designer can design a system that takes full advantage of partial reconfiguration, using a library of components developed by RTL designers, without needing experience in low-level FPGA details.

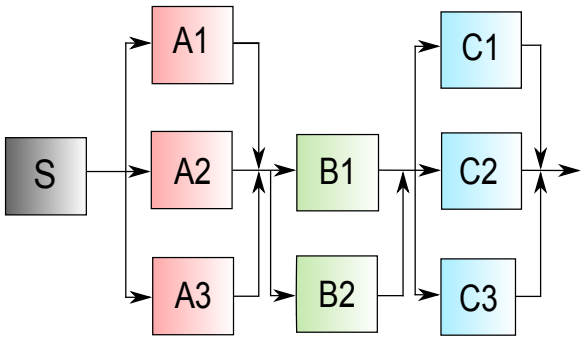


Figure 3.1: Example PR design.

3.2 Partial Reconfiguration Tool Flow

It is first important to define the terms we use in this paper. A PR *region* is an area on the device allocated to logic, that can be reconfigured at runtime. It includes different types of basic primitives as exist on the FPGA fabric and information about each region is required by the partial reconfiguration tools. A *module* is a processing unit in the design and may have multiple *modes*. In this discussion, modes are mutually exclusive implementations of the module with the same set of inputs and outputs. At runtime, we may wish to switch a module from one mode to another. A *configuration* is a set of possible co-existent modes for all the modules in the system. Since not all possible combinations of module modes will be valid, this allows us to focus on solutions that are optimal for the valid sets of modes. If we were to consider all possible combinations, the number of scenarios would grow exponentially with the number of modes per module. If we have 4 modules, each with 3 modes, we would need $3^4 = 81$ possible combinations. However, if we know that some modes do not co-exist, we cut down the number of global configurations significantly. This means that our solution is optimised for those global configurations that may arise, rather than configurations that never will.

For efficient implementation and management, a hierarchical module based design approach should be followed for PR designs [111]. Fig. 3.1 shows an example PR design. The first task is to divide the design into static logic and reconfigurable modules. The functionality of the static logic does not change during FPGA operation.

There can be one or more reconfigurable modules. In Fig. 3.1, module S represents the static logic and modules A, B, and C represent reconfigurable modules. In the example, A_1, A_2, and A_3 are different modes of reconfigurable module A. Reconfigurable modules are implemented in reconfigurable regions, typically each module mapping to a distinct region. The regions contain FPGA resources such as Slices, BlockRAMs, DSP Slices, etc. The designer should ensure that each region contains sufficient resources to implement all modes of the modules assigned to that region. This requires FPGA architecture knowledge and manual floor-planning. The designer must them use the implementation tools to build netlists for all modes for all regions.

For the example design given in Fig. 3.1, $S \rightarrow A_1 \rightarrow B_1 \rightarrow C_1$ is a possible configuration. Each configuration contains the static logic and one of the possible modes for each reconfigurable module. In the current tool flow, configurations do not play any role in synthesis since the reconfigurable modules and the assignment of regions, are performed manually. The designer will prepare netlists only for valid combinations of module modes in each region. Hence, it is clear that partial reconfiguration, as supported currently, is a floorplanning activity. We wish to bridge the gap, such that it becomes a system design feature that can be used by people who do not understand floorplanning. The first step is to determine how to allocate regions.

Before discussing how to optimise region allocation, we should understand the costs we are trying to minimise. The size of a partial bitstream is proportional to the size of the region and determines the reconfiguration time for that region. Each time a module reconfigures, the whole region to which it is assigned must be reconfigured. Hence, while combining modules into fewer regions can allow the tools to optimize resource usage, it is clear that the reconfiguration time can increase dramatically. This work seeks to determine an allocation that results in as small an area requirement as possible, and as short an average reconfiguration time as possible, taking into account the costs of allocation decisions in terms of both area and reconfiguration time.

3.3 Problem Formulation

3.3.1 Fundamentals

From Section 3.2, it is clear that the current supported PR tool flow requires extensive input from the designer, who is expected to know the details of the target FPGA architecture, as well as preparing all region netlists. We formulate an analytical model, which can integrate with the existing PR tool flow in order to generate efficient PR designs without detailed input from the designer. Given an application description this tool will define the optimal number, and size, of partial regions and the assignment of modules to those regions. It will then pass this information and the resulting netlists to the PR implementation tools.

The simplest way to partition the FPGA for partial reconfiguration is to divide the whole FPGA into two: one static region and one PR region. All the static logic in the design is implemented in the static region, while modules that require reconfiguration are implemented in the PR region. This approach has some benefits. Firstly, the designer only needs to allocate a single region, large enough to hold the most resource hungry configuration. Secondly, the tools can optimise area usage and timing across all modules resulting in the best possible timing performance and area.

However, some major drawbacks mean this method is not ideal. Firstly, each time any module in the region needs to be reconfigured, the whole region must be reconfigured, and so the frequency of reconfiguration will be the sum of the frequencies of reconfiguration for all modules. Secondly, since the whole region must be reconfigured even if a small module is being changed, the reconfiguration time is increased, in some cases significantly. Finally, designs with many possible combinations of modes will require a large bitstream for each possible configuration, resulting in significant storage being required to store bitstreams. Hence, simply allocating all reconfigurable modules to a single region is not ideal for systems that rely on minimising reconfiguration time and bitstream storage.

Reconfiguration time can in some cases be the key requirement for an application. Take for example a cognitive radio system that is reusing spectrum. If a primary user appears, the radio must immediately cease sending and search for empty spectrum.

Similarly, a driver assistance system should adjust between different scene processing modes in as short a time as possible. Video applications may require reconfiguration to take place in the inter-frame interval. Hence, it is important to consider both worst-case and average reconfiguration times, and determine application constraints.

Generally, a one-region-per-module approach will offer the lowest worst-case reconfiguration times, since a region will only reconfigure when its sole module needs to, and the size of the region is only as large as the largest mode of that single module. However, a one-region-per-module approach is the least area-efficient way to allocate regions.

An analytical approach would allow us to determine how many regions to use, and which region each module should be allocated to, with a view to minimising reconfiguration time, while also maintaining area efficiency.

System configurations play an important part in the following discussion. For most systems, not all combinations of module modes will be encountered at run time. In fact, there are normally a small number of possible combinations, that are referred to as configurations. Configurations greatly reduce the search space, since we only need to consider possible mode combinations to arrive at an optimal allocation. If we make the assumption that all possible combinations of modes for all modules are allowed, this means that combining modules into a single region, we require as many bitstreams for that region as the cartesian product of their numbers of modes. It also means we have no way of determining how a certain grouping of modules impacts the reconfiguration time, as we have no idea which modules are more likely to be reconfigured. Instead, by using configurations — sets of allowable mode combinations — we limit the cost of combining modules into regions, and allow further analysis of the reconfiguration time, as will be discussed in Section 3.3.3.

One way of representing configurations is with an adjacency matrix. Each dimension represents a module, with its corresponding modes. Each position in the matrix indicates whether that combination of modes exists in any valid configuration. This is easy to see for two modules. Consider module A, with modes $A_1 \cdots A_n$ and another module B with modes $B_1 \cdots B_m$. The adjacency matrix $\mathrm{A_{A,B}}$ is an $n \times m$ matrix, as shown.

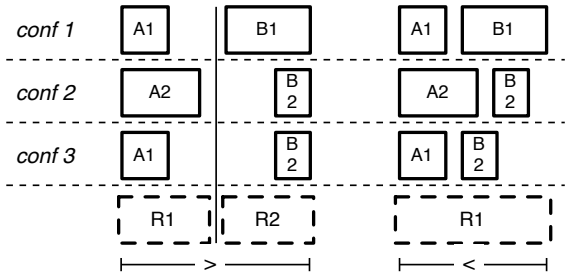


Figure 3.2: When assigning modules to separate regions, if some configurations do not exist, combining modules into a single region can save area.

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$

This matrix indicates that a configuration exists with module modes A_1 and B_1 but A_1 and B_2 never coexist. Similarly A_2 and B_2 coexist but A_2 and B_1 do not. If the adjacency matrix between two modules is an identity matrix, it means that for each mode of the first module, there is a corresponding mode of the second module. In this case, the two modules should be allocated to the same region, since they can be optimised together and always reconfigure together, meaning there is no additional overhead in terms of configuration time or storage of bitstreams.

If the adjacency matrix is not an identity matrix, the decision is not as straightforward and would depend on the cost of combining the modules into a single PR region. Consider two modules, each with a large and small mode, as shown in Fig. 3.2. If they are allocated to separate regions, the regions must each be large enough for the largest mode of the corresponding modules. However if we know that the largest mode for both never exist together, then if they are combined in a single region, that region only needs to be large enough for the largest overall configuration.

Unfortunately, beyond two modules, the adjacency matrix becomes multi-dimensional and hard to interpret, hence a mathematical formulation of the above problem is more appropriate, allowing multiple modules to be considered simultaneously, and a globally optimal solution found.

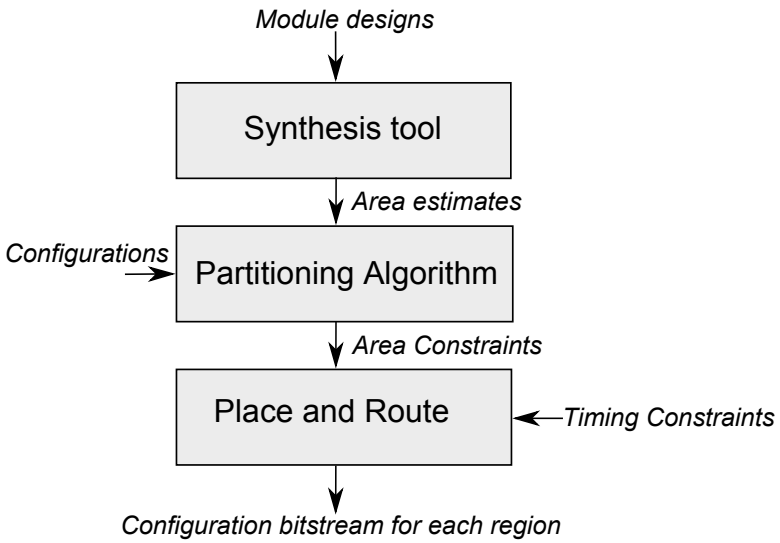


Figure 3.3: Proposed tool flow.

3.3.2 Proposed Tool Flow

Our proposed tool flow for solving this PR region allocation problem is shown in Fig. 3.3. The designer provides design files for all modules (in all modes), a list of allowable configurations and design implementation constraints such as timing constraints to the automation script in XML format. The script performs the following steps.

1. Vendor supplied *XST* tool is used for synthesising all the modules in order to extract the resource requirement information of all modes of modules. Alternatively resource estimation techniques can also be used [112]. If IP cores are used for some modules, usage information is often available up front.
2. The system specification is updated in the XML representation with resource requirements of different modes of modules. An example snapshot of the XML specification is shown in Fig. 3.4.
3. The resource requirements are passed to the equation solver with a list of valid configurations. It uses these inputs with the formulation discussed in the next section, and determines the optimal region configuration.
4. Once a region allocation has been decided, wrapper modules are created that represent different configurations for each region.

```
<system_specification>
  <number_of_modules>5</number_of_modules>
    <module name="module1">
      <nummodes>2</nummodes>
        <mode label="1">
          <SLICE>818</SLICE>
          <BR>0</BR>
          <DSP>28</DSP>
        </mode>
        <mode label="2">
          <SLICE>500</SLICE>
          <BR>0</BR>
          <DSP>34</DSP>
        </mode>
    </module>
    <configurations>
      <number_of_configurations>2</number_of_configurations>
        <conf label="1">
            <instance>module1.1</instance>
            <instance>module2.2</instance>
        </conf>
    </configurations>
</system_specification>
```

Figure 3.4: XML based system specification.

5. A netlist for each wrapped configuration is then automatically generated using vendor synthesis tools.
6. Determining the floorplanning of the static and reconfigurable regions must then be done. A number of floorplanners are available that are tailored for partial reconfiguration [44], and these may be used. Alternatively, a manual approach using Xilinx *PlanAhead* is possible.
7. The area constraints generated by the floorplanner along with the timing requirements are used for generating the *User Constraints File*, which directs the place and route tool to achieve the design goals. The netlists generated, as well as the User Constraints File, are passed to PlanAhead, which performs the placement and routing operations.

3.3.3 Mathematical Formulation

To solve this problem, we represent it mathematically using an objective function and a number of constraints. Based on the previous analysis, the problem of finding

the optimal number of regions and the region that each module should be assigned to can be described using the objective functions:

1. Minimise average reconfiguration time.
2. minimise the total FPGA resource requirement.

Subject to the conditions:

1. All modules in the design being implemented,
2. all required configurations being implemented,
3. each module being implemented only once,
4. design fitting in the given device,
5. the number of PR regions should be greater than or equal to 1 and less than or equal to total number Reconfigurable Modules.

We denote the variables used in the formulation as shown in Table 3.1.
The maximum number of resource type i for module u is given by the maximum usage of i in different modes of u:

$$R_{uiMAX} = \max_{m}(R_{umi}), \tag{3.1}$$

Since each module should be implemented only once, the sum of allocation decision variables should be 1:

$$\sum d_{uq} = 1. \tag{3.2}$$

If multiple reconfigurable modules are merged into a single region, the area required for resource type i for each configuration c is determined as follows. The area required for each mode of module u is taken into account only if the mode exists in the current configuration c. The area required for each module is summed over all modules present in region q. The partition method can vary from using a single region to using separate region for each module.

$$R_{qic} = \sum_{u} R_{umi} * d_{umc} * d_{uq}; c = 1, 2, ...C; q = 1, 2....N \tag{3.3}$$

Table 3.1: Notation used in formulation.

Notation	Meaning
F_i	Total amount of resource type i in the FPGA (types can be Slice, BRAM, DSP)
N	Total number of reconfigurable modules
C	Set of Configurations
R_s	Reconfiguration speed
d_{uq}	Decision variable – 1 if module u is present in reconfigurable region q otherwise 0
d_{umc}	Decision variable – 1 if module u is present in mode m in configuration c otherwise 0
R_{uiMAX}	Maximum number of resource type i used by module u
R_{umi}	Number of resource type i used by module u in mode m
R_{di}	Total requirement of resource type i in the partition scheme
R_{qic}	Number of resource type i used in region q in configuration c
R_{qi}	Maximum number of resource type i consumed by region q
A_q	Area of region q in normalized units
W_i	Area weighing factor for resource type i
W_{fi}	Number of frames in resource type i
t_q	Reconfiguration time for region q
t_c	Reconfiguration time for configuration c
t_w	Worst case reconfiguration time
t_a	Average reconfiguration time

From the set of resource requirements for different configurations, the maximum resource requirement for type i is determined, which is the required result.

$$R_{qi} = \max_{c}(R_{qic}) \tag{3.4}$$

The total number of resource type i required for the complete design is the sum of resource type i among all the regions

$$R_{di} = \sum_{q} R_{qi} \tag{3.5}$$

In order for the design to fit into a particular FPGA, for each resource type i, the total resources required should be less than or equal to resource type i present in the

FPGA.

$$F_i - R_{di} \geq 0 \tag{3.6}$$

The total area cost of region q is given by

$$A_q = \sum_i W_i * R_{qi} \tag{3.7}$$

where W_i is the weighing factor for resource i calculated as the ratio of total resources to resources of type i.

Now consider reconfiguration time. Reconfiguration time for region q can be determined by dividing the area of q by the speed of configuration.

$$t_q = \sum_i W_{fi} * R_{qi}/R_s, \tag{3.8}$$

where W_{fi} is the weighing factor determined by the number of reconfigurable frames required for resource type i. When modules are merged, reconfiguration of any of the modules in the region will lead to the reconfiguration of the whole region. The frequency of reconfiguration is application dependent. Total configuration time for the system when it changes from configuration c_i to configuration c_j is calculated as the sum of the configuration time for regions, whose modules change their modes.

$$t_c = \sum_q t_q * d_{cq}, \tag{3.9}$$

where $d_{cq} = 1$ if, for any $d_{uq} = 1$, $d_{umci} \neq d_{umcj}$ else 0. Average reconfiguration time is the average of all possible configuration times.

$$t_a = \bar{t_c} \tag{3.10}$$

Worst case reconfiguration time (t_w) for a partition scheme is calculated as the maximum reconfiguration time among all possible configuration transitions.

$$t_w = \max(t_c) \tag{3.11}$$

In order to improve the overall system performance, average reconfiguration time is selected as the minimisation objective. For applications where a strict reconfiguration time limit must be met, worst case reconfiguration time can be selected as the objective function.

3.3.4 Integer Linear Programming

The exact solution for this problem can be found using Integer Linear Programming (ILP). Although ILP is known to be NP-Complete, the number of reconfigurable regions in an adaptive system will typically be 10 or fewer. In this case, ILP can solve the problem effectively and can provide a globally optimal solution. In order to solve the problem using an ILP solver, it needs to be represented in a specific format having an objective function and number of constraints. From the above problem formulation, ILP equations can be represented as

$$Minimise(\sum_{q} A_q), \tag{3.12a}$$

$$Minimise(t_a). \tag{3.12b}$$

subject to

$$\sum d_{uq} = 1, \tag{3.13a}$$

$$A_q = \sum_{i} W_i * R_{qi}, \tag{3.13b}$$

$$R_{di} = \sum_{q} R_{qi}, \tag{3.13c}$$

$$F_i - R_{di} \geq 0, \tag{3.13d}$$

$$t_c = \sum_{q} t_q * d_{cq}, \tag{3.13e}$$

$$t_a = \bar{t_c}. \tag{3.13f}$$

These equations can be solved by freely available ILP solvers such as LPSolve [113]. The values of W_i, W_{fi} as well as the resource availability, are FPGA dependent.

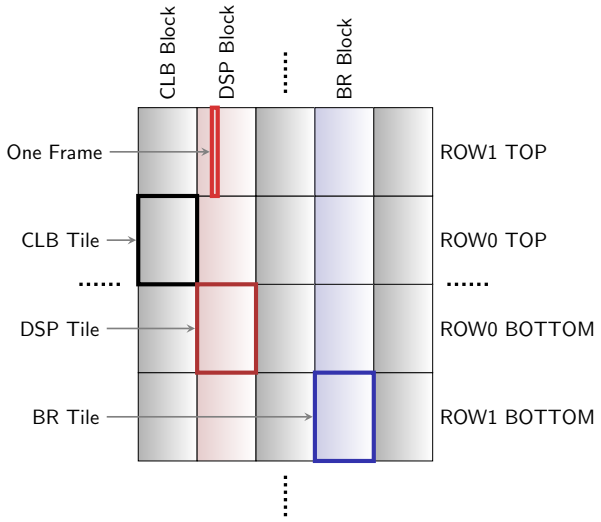


Figure 3.5: Virtex 5 FPGA architecture.

These can be stored in a database file and the solver can be pointed to a particular FPGA in order to find an optimal partition for that FPGA, or to determine the most suitable FPGA device for the application.

3.4 Optimal Region Allocation

3.4.1 Architecture Analysis

In Xilinx Virtex FPGAs, the smallest reconfigurable unit is a *frame*. In Virtex-5 FPGAs, the height of a frame spans an entire device row [23]. The number of rows[1] in a device depends upon the size of the device. In the Virtex-5 FX70T, which contains 11,200 Slices, 128 DSP Slices and 296 BlockRAMs, there are eight rows. A row crosses columns of different resource types, including Configurable Logic Blocks (CLBs), DSP Slices and Block RAMs. These columns extend the full height of the device and are referred to as *blocks*. A tile is one row high and one block wide, and contains a single type of resource as shown in Fig. 3.5. One CLB tile contains 20 CLBs, one DSP tile contains 8 DSP Slices and one BRAM tile contains 4 Block RAMs. For

[1]Note "rows" here refers to a Xilinx term used in their architecture descriptions.

accurate calculation of efficient partial reconfiguration schemes, the reconfigurable regions should be considered in terms of these basic tiles since configuration must occur on a per tile basis. Even though regions may include partial tiles, extra circuitry is needed to manage reconfiguration and the time taken is increased in such cases. Hence in all equations, the resource i represents one tile of resource type i (Slice, DSP, BRAM, etc). The FX70T device contains 280 CLB tiles, 16 DSP tiles and 74 Block RAM tiles. A general measure of area cost for each region is found by normalising the area for CLB tiles, DSP tiles and BRAM tiles. The normalisation factor used is 1:18:4 for CLB, DSP and BRAM tiles respectively, based on the number of tiles in the device, which corresponds to W_i in the formulation. One CLB tile contains 36 frames, DSP tile 28 frames and BRAM tile 30 frames, which corresponds to W_{fi} in the formulation.

3.4.2 Application Results

Here, we apply our region allocation approach to an example design implemented on a Virtex-5 FX70T FPGA. The design has one static region and five reconfigurable modules. The design is a wireless video receiver chain, which contains a matched filter, signal recovery, demodulation, channel decoding and video decoding with blocks used from existing designs and IPs. The system can operate in various modes, and adapts to channel conditions and user requirements at runtime. The resource utilisation for each reconfigurable module and mode is as shown in Table 4.1. The configurations used are

1. $S \rightarrow F_1 \rightarrow R_3 \rightarrow M_1 \rightarrow D_1 \rightarrow V_1$
2. $S \rightarrow F_1 \rightarrow R_3 \rightarrow M_1 \rightarrow D_1 \rightarrow V_2$
3. $S \rightarrow F_1 \rightarrow R_3 \rightarrow M_1 \rightarrow D_1 \rightarrow V_3$
4. $S \rightarrow F_2 \rightarrow R_1 \rightarrow M_2 \rightarrow D_3 \rightarrow V_1$
5. $S \rightarrow F_2 \rightarrow R_1 \rightarrow M_2 \rightarrow D_3 \rightarrow V_2$
6. $S \rightarrow F_2 \rightarrow R_1 \rightarrow M_2 \rightarrow D_3 \rightarrow V_3$
7. $S \rightarrow F_2 \rightarrow R_2 \rightarrow M_1 \rightarrow D_1 \rightarrow V_1$
8. $S \rightarrow F_2 \rightarrow R_2 \rightarrow M_1 \rightarrow D_1 \rightarrow V_2$

Table 3.2: Resource utilisation for reconfigurable modules.

Module	Mode	Slices	BR	DSP
Matched Filt (F)	1. Filter1	818	0	28
	2. Filter2	500	0	34
Recovery (R)	1. Fine	318	1	13
	2. Coarse1	195	1	5
	3. Coarse2	123	0	8
	4. None	0	0	0
Demodulator (M)	1. BPSK	50	0	2
	2. QPSK	97	0	4
Decoder (D)	1. Viterbi	630	2	0
	2. Turbo	748	15	4
	3. DPC	234	2	0
Decoder (V)	1. MPEG4	4700	40	65
	2. MPEG2	4558	16	32
	3. JPEG	2780	6	9

9. $S \rightarrow F_2 \rightarrow R_2 \rightarrow M_1 \rightarrow D_1 \rightarrow V_3$
10. $S \rightarrow F_1 \rightarrow R_4 \rightarrow M_2 \rightarrow D_2 \rightarrow V_1$

This system description was formulated as in the previous section, and the ILP solver used to find the optimal solution. To obtain an estimate of average reconfiguration time for different schemes, a transition matrix, T is used. For a system with n configurations, the transition matrix is an $n \times n$ matrix, where each element $T(i)(j)$ represents the possibility that the system will transition from configuration i to j. A value of 1 means the system can switch to configuration j from configuration i; a 0 indicates that such a transition is impossible. In practical cases, T is application-dependent and something that the adaptive system designer would need to determine. A more accurate model can be found by probabilistic analysis of the deployed system, resulting in a transition matrix in which each element $T(i)(j)$ will represent the probability of system switching from configuration i to j. The average reconfiguration time is found by averaging the configuration time for all configuration transitions after multiplying with corresponding $T(i)(j)$ coefficientss. In the current design, the transition matrix used is

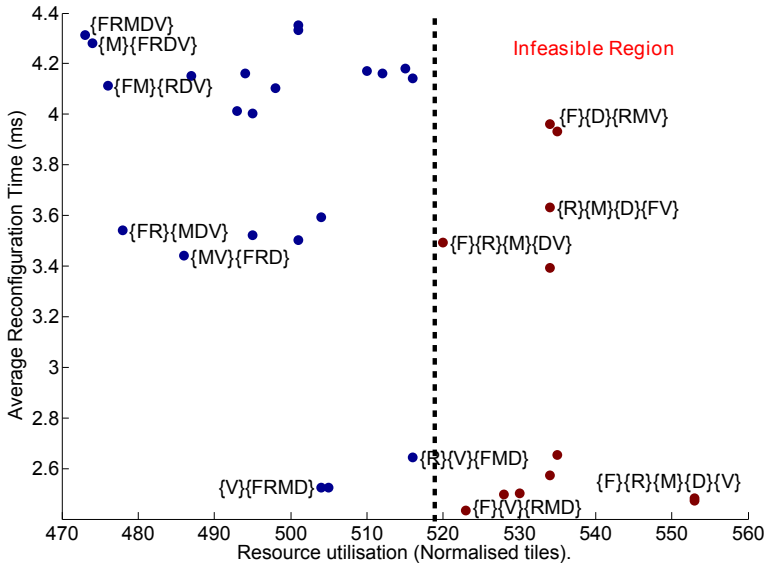


Figure 3.6: Resource requirement and configuration time.

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

A plot of total resource requirement and average reconfiguration time is shown in Fig. 3.6. Reconfiguration speed is taken as 234 MB/s [114]. Several of the solutions are infeasible due to lack of resources in the FPGA. Using a one-region-per-module scheme, this design will not fit into the FX70T device, since that scheme requires 18 DSP tiles and the device has only 16. Using a larger FPGA would increase system cost significantly. The partition scheme in which all the modules are implemented

together (labelled {FRMDV}) gives the lowest resource utilisation of 473 tiles, but the average reconfiguration time for this scheme is considerably higher at 4.32 ms. Using the proposed partitioning method, we can find schemes that fit the design into the FX70T device with minimum reconfiguration time overhead. This is one of the attractions of using this analytical method. There are six Pareto optimal points in the feasible region of the plot. The configuration in which the decoder (V) is implemented in a single region and all other modules are implemented together in another region, labelled {V},{FRMD} in the plot, gives the lowest average reconfiguration time. This scheme uses 504 normalised tiles and has a average reconfiguration time of 2.52 ms. The scheme in which the filter (F) and recovery (R) modules are combined in a single region and other modules are implemented together (labelled {FR},{MDV} in the plot) lies closest to the origin and hence is the optimal partitioning. This scheme uses 478 normalised tiles and the average reconfiguration time is 3.54 ms and hence gives 13.5% area improvement compared to one-module-per-region partitioning. Worst case reconfiguration time for the optimal scheme is 4.36 ms, that for one-region-per-module partitioning is 4.69 ms. The bitstream storage requirement for these partitions was also calculated. The optimal solution {FR},{MDV} requires 53 Mbits storage while implementing all modules in a single region requires 81 Mbits to store bitstreams. These results depend significantly on the configurations defined by the application. The upper bound area consumption will be that of using separate regions for each module and the lower bound is a single-region scheme.

3.5 An Improved Partitioning Algorithm

In this section, we introduce an improved partitioning algorithm from the experiences gained from our previous experiments. This algorithm removes constraints such as all module modes should be implemented in a single region. It is based on heuristics, which removes the requirement of integer programming.

For explaining the algorithm, consider the example design given in Fig. 3.1. As described previously designers generally adopt two simple methods for partitioning PR designs. First is to assign all modules to a single region and the second is to assign

each module to a separate region. Assigning all modules to a single region consumes the least amount of hardware, since the resource requirements in this case is that of the largest configuration. Now consider two modules, A and B, for an example. Each module has a large and a small mode, as shown in Fig. 3.2. The three system configurations are $\{A_1, B_1\}$, $\{A_2, B_2\}$, and $\{A_1, B_2\}$. The scheme with all the module modes implemented concurrently requires minimum reconfiguration time as well as storage. This is equivalent to the complete static implementation of the system. The total resource consumption will be the sum of the resource requirements of each mode. In this scheme, the reconfiguration time is a few clock cycles required to switch the multiplexers to change the module modes. Since the system is completely static, no partial bitstreams need to be stored and used during a system reconfiguration. This method is usually impractical due to the limits imposed based on FPGA resources for the complete static implementation of the design. It would require the designer to target a larger, more costly FPGA. This is even more apparent when the number of modes is high.

If each module is assigned to a separate region, each region must be large enough for the largest mode of the corresponding module assigned to it. In the example, there will be two regions, one as big as A_2 and the other as big as B_1 and hence the total resource requirements is the sum of the resource requirements of these two modes. Any system reconfiguration will require configuring atleast one reconfigurable region, and in one case both ($\{A_1, B_1\}$ to $\{A_2, B_2\}$). Four partial bitstreams, corresponding to each mode, need to be stored, and the size of the bitstreams is proportional to the size of the regions.

However if the modules are combined in a single region, that region only needs to be large enough for the largest overall configuration. In this case the size of the single region will be that of $\{A_1, B_1\}$. The is because the largest modes of the two modules (A_2 and B_1) do not co-exist among the possible system configurations. This scheme gives the lowest resource requirement. Here the drawbacks are, three partial bitstreams, each as big as $\{A_1, B_1\}$, need to be stored corresponding to each system configuration. Any system reconfiguration requires reconfiguring the entire region, which increases the reconfiguration time overhead.

Finally consider a hybrid case. Suppose A_2 and B_1 are implemented in a single region and A_1 and B_2 in separate regions. The total resource consumption in this case will be the sum of B_1, A_1 and B_2. Although there are three regions, a close examiniation will reveal that the resource consumption in this case is almost similar to that of assigning each module to a separate region, and much less than complete static implementation. Here $A1$ and B_2 are implemented as static logic, hence no partial bitstreams are needed for them. Two partial bitstreams for A_2 and B_1 have to be stored. The number of reconfiguration operations required is also lesser in this scheme, since A_1 and B_2 are in static logic. This simple example emphasises the importance of considering configurations during partitioning.

Since runtime relocation of bitstreams is impractical for latest FPGAs due to their heterogeneous architecture and disparate arrangement of resources, all the required bitstreams need to be pre-generated and stored. The amount of bitstreams that can be stored is limited by the system parameters. Although adaptive systems are dynamic in nature, they can operate only in configurations specified by the configuration manager, usually software running on a processor, which controls loading and unloading of modes based on environmental changes. Hence the system designer is aware of the configurations in which the system will operate, although it may be impossible to determine the order in which they system will switch the configurations.

The major constraints associated with PR design are resource availability in the target FPGA, reconfiguration time and memory for storing configurations. A system desginer can specify the configurations of the system, targeted FPGA, and the maximum storage available for bitstreams. Based on this, the tool needs to generate a partitioning, which will fit into the target FPGA and does not cross the constraints boundary.

So the partition problem can be modelled as: Given

1. an FPGA with resource availability,
2. the number of modules and their modes,
3. the possible configurations,
4. the maximum system storage available for storing configurations,

partition the design in such a way that

1. The design fitting into the FPGA,
2. all configurations being implemented,
3. all configuration transitions being possible,
4. bitstream storage limitations being satisfied,
5. reconfiguration time being minimised.

Algorithm

As the first step, a connectivity graph is generated based on the configurations. A connectivity graph *G(V,E)* is a graph, whose vertices V represent the modes and edges E represent the number of times the connected modes coexist in the configurations. For the example design introduced in section 3.2, the connectivity graph is as shown in Fig 3.7. An edge value of 3 between modes B_2 and C_1 indicates that, B_2 and C_1 co-exist in three of all possible configurations. A larger edge value represents that the two connected modes are configured together more frequently, and hence they are more suitable for configuring in the same region.

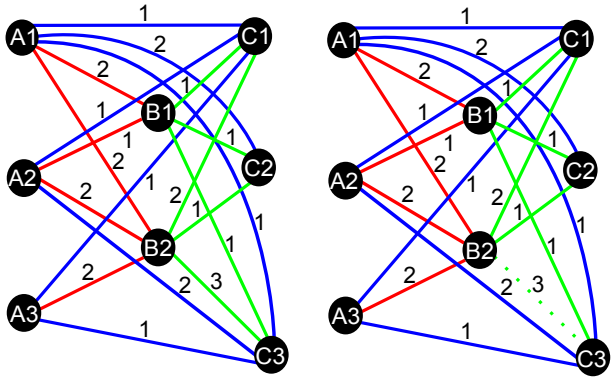


Figure 3.7: Example of connectivity graph. Once the modes with highest connectivity are merged, corresponding edges are removed from the graph

From the connectivity graph, *base partitions* are generated. A *base partition* is a partition of modes, which is used for generating the final partition. Inorder to generate *base partitions*, an iterative algorithm is used, which selects the largest edge

value from the connectivity graph. The nodes connected by this edge are considered as parent nodes and the nodes connected to the parent nodes are child nodes. Parent nodes are merged to create a *base partition* and the child nodes are merged to this partition, if the connectivity values of the child with all the parent nodes are equal to the connectivity between the parent nodes. In the graph shown in Fig.3.7, the algorithm initially selects modes B_2 and C_3, since their connectivity value is the maximum (3). Then B_2 and C_1 become the parent nodes and the child nodes connected to both of them are A_1, A_2 and A_3. None of these child nodes have connectivity equal to 3. Hence none of the child nodes are merged with the parent nodes, and hence the first *base partition* becomes $\{B_2,C_3\}$. Once a *base partition* is created, edge value between the modes present in the partition are replaced with a value of 1, and edge value of the parent nodes are stored as the $frequency\ weight$ of the *base partition*. For the *base partition* $\{B_2,C_3\}$, the $frequency\ weight$ is 3. If the edge value between the parent nodes is 1 when *base partitions* are generated, that edge is deleted after the $frequency\ weight$ of the *base partition* is stored. This algorithm iterates until all edges in the graph are deleted. Once all edges are deleted, only the nodes remain, which represent all the modes. Each of these nodes is also added into the list of *base partitions* with a $frequency\ weight$ of 1.

Once *base partitions* are generated, a covering algorithm is used for selecting the *base partitions* used for final partitioning. For this purpose, *base partitions* are arranged in a list with ascending order of number of modes present in them. If two *base partitions* have same number of modes, they are arranged ascending order of $frequency\ weight$. *Base partitions* with same $frequency\ weight$ are arranged in the ascending order of their resource requirements. Configurations and modes are arranged in a matrix format with each row representing a configuration and each column representing a mode. A value of 1 of element (i, j) in the matrix represents mode j is present in configuration i. Now *base partitions* are selected from the list in a sequential order and compared with the matrix. For each configuration, the corresponding modes present in the selected *base partition* are set to zero in the configuration matrix. Subsequently *base partitions* $\{C_2\}$ and $\{C_3\}$ are used for covering more configurations. *Base partitions* are selected and compared from the

list until all elements in the matrix become zero, and set of *base partitions* used for covering the entire configurations become the *final partition* set. The rationale for this operation is that, in order to implement all configurations, there should be enough base partitions covering all the modes.

Next step is alloting the partitions to regions. For this purpose, the tool finds the compatible set of partitions for each *base partition* from the *final partition* set. Two partitions are compatible if the modes present in them do not co-occur in any of the configurations. The operation is necessary to make sure that all configurations can be implemented. If two base partitions required for a single configuration are alloted to the same region, that configuration cannot be implemented since at a given instance, only one base partition will be active in a configurable region. Region allocation starts with alloting each *base partition* to a separate region, which makes an initial partitioning. This initial assignment is equivalent the complete static implementation of the system. The total resource requirement, average reconfiguration time and storage requirement for this partitioning is calculated. After this, compatible *base partitions* with least area variance are assigned to same region. The total area of each region is the area of the largest *base partiton* assigned to that region. The comparison matrices such as resource consumption, reconfiguration time etc. are again calculated. If the resource consumption for a *final partition* is more than the resource or storage availability, it is immediately rejected.

Once all possible compatible *base partition* assignments are done, a new set of *base partitions* is selected from the list for generating new set of final partitions. For this purpose, the topmost *base partition* is removed from the list, and covering algorithm is performed again. The new set of *base partitons* considered for the final partition. If a *base partition* cannot cover any new mode, it is not considered for final partitioning, since not covering any new mode indicates that this *base partition* is superseded by previous *base partitions*.

This alrogithm makes sure that different types of partitioning, with varying number of reconfigurable regions, are evaluated. The number of regions vary from the total number of modes to a single region. The automatically evaluated partitions include assigning each mode to separte region, merging modes to implementing all modes

in a single region. Considering the configuration information in the partitioning step makes it a tractable problem, where as if all possible combinations of modes were considered, the problem will become NP-hard and solution will be sub-optimal. From the set of final partitions, Pareto-Optimal points are found considering the resource requirement and reconfiguration time and are used for the final implementation.

3.6 Conclusion and Future Work

Determining the number of partial reconfiguration regions and the allocation of reconfigurable modules to regions is not always trivial, but this choice can impact FPGA resource utilisation, reconfiguration time and the storage requirement for configuration bitstreams. We have introduced a new technique, which can be incorporated into the existing vendor-supported partial reconfiguration tool flow for automating the partitioning step. It is demonstrated that efficient reconfiguration schemes improve resource consumption and reduce reconfiguration time. The method presented here will help designers with less FPGA architecture knowledge to generate efficient partial reconfiguration implementations, allowing designs to meet limited device constraints. Inorder to reduce reconfiguration overhead, the algorithm can exploits prefetching capabilities. Additional bitstreams can be generated, until the total storage requirements reaches the ceiling specified by the designer. During runtime, the reconfiguration management software controls the prefetching operation. Since the order in which the system will change its configuration is unknown, the configuration management software can prefetch the *base partition* with the highest $frequency\ weight$ to each region, which are not used in the current system configuration. One important future research direction is automatically incorporating the partition information and runtime management of loading and unloading of modes into the configuration management software. Case studies have to be performed on the newly proposed partitioning algorithm. We are currently working on that. Through our ongoing research, we are intending to generate a high level abstraction of partial reconfiguration process, so that this technique can be easily adopted by adaptive system level engineers with confidence. This level of abstraction requires development of efficient

methods for partitioning, region allocation, floorplanning, communication interface development and application specific library generation.

3.7 Related Publication

The work in Sections 3.3 and 3.4 was published in the following paper:

1. Kizheppatt Vipin, Suhaib A Fahmy, *Efficient Region Allocation for Adaptive Partial Reconfiguration*, Proc. of IEEE International Conference on Field Programmable Technology (FPT), New Delhi, 2011.

Chapter 4

Floorplanning PR Designs

4.1 Introduction

For standard static FPGA design, floorplanning is generally only of interest to expert designers trying to highly optimise a design. The tools available from vendors are sufficiently versatile to perform area-constrained placement and routing, while achieving timing closure. Current PR tools do not perform floorplanning automatically, and require considerable input from the designer. The designer is expected to have knowledge about the physical architecture of the target FPGA, as well as the details of the PR process and the run-time costs associated with it, if they are to come up with an efficient floorplan. Manual floorplanning based on these factors is cumbersome and often leads to sub-optimal results and consumes a large amount of design time. This floorplanning requirement has made PR less attractive to adaptive system designers. An intelligent arrangement and allocation of PR regions can result in reduced area and hence allow designs to fit on smaller devices. We present an approach that optimises the runtime properties by finding a placement that results in the lowest possible reconfiguration time, considering the lowest level granularity of heterogeneous resources on modern FPGAs, for designs where the adaptation is at a functional level and hence unpredictable.

4.2 Contributions

In this section we propose methods, which can help system engineers adopt PR without the need for manual floorplanning. The tool we propose can be integrated into the existing FPGA tool chain. We are considering adaptive applications where reconfiguration occurs at the module level, and the sequence of configurations is unknown up front. We consider the overheads associated with PR as well as the characteristics of target devices. We are interested in recent families of FPGAs such as the Xilinx Virtex-5 and Virtex-6, which are highly heterogeneous in nature, include embedded processors and transceivers, and comprise an irregular arrangement of DSP and Block RAM columns. For PR applications, we are typically concerned with reducing reconfiguration time and area. Cost functions are used that take into account several factors such as resource wastage and reconfiguration time, rather than ASIC floorplanning metrics. The main contributions are:

1. A detailed analysis and presentation of factors that affect the efficiency of floorplans for PR designs.

2. A novel method for floorplanning on modern heterogeneous FPGA architectures, that improves PR design characteristics.

3. A comparison of the proposed floorplanning efficiency with existing approaches.

4.3 PR Floorplanning Considerations

In order to unburden the designer from manual floorplanning, the complexities of the floorplanning task should be borne by the tool. In this section we develop a device model and explore the factors to be considered while designing an efficient floorplanner for PR. The limitations of several existing methods will be also explained.

4.3.1 Architecture Considerations

For efficient floorplanning, the tool should be aware of the FPGA architecture and special requirements arising due to PR. Xilinx Virtex-5 FPGAs are divided into rows

and columns. The number of rows in a device depends upon the size of the device. The smallest configurable unit is a *frame*. A *frame* is one bit wide and spans an entire device row [23]. Resources such as CLBs, Block RAMs etc. are arranged in a columnar fashion extending the full height of the device and are referred to as *blocks*. A tile is one row high and one block wide, and contains a single type of resource, as shown in Fig. 3.5. One CLB tile contains 20 CLBs, one DSP tile contains 8 DSP Slices, and one BRAM tile contains 4 Block RAMs arranged vertically. A CLB tile contains 36 *frames*, a DSP tile 28 and a Block RAM tile 30 *frames* arranged side by side. The data size of a $frame$ is 164 bytes. Embedded processors and transceivers are arranged along device row boundaries, but they obstruct the continuity of DSP and BRAM columns. Most existing floorplanners have not taken this device architecture into account, claiming only to use regular grid structures.

4.3.2 PR Operation

Partial reconfiguration is performed by modifying the circuitry implemented in the PR regions. Any modification in a region requires the full reconfiguration of the corresponding region. For accurate calculation of efficient PR schemes, the reconfigurable regions should be considered in terms of tiles since configuration must occur on a per tile basis. To use regions with incomplete tile boundaries, extra circuitry is required to read, modify, and write configuration information, resulting in increased area and latency. Reconfigurable regions must always be rectangular in shape. Since each tile is one device row high, the height of reconfigurable regions is an integer multiple of device rows. Partial bitstreams are loaded into the FPGA with the help of an Internal Configuration Access Port (ICAP), which is usually controlled by an embedded processor. The size of the bitstream, and hence the reconfiguration time of a region, is directly proportional to the total area of the region, irrespective of how many resources present in the region are actually utilised.

4.3.3 Required Reconfigurable Area

An important factor in floorplanning a reconfigurable region is its area. At runtime, reconfigurable regions implement different functional instances at various points in time. These functional instances are called *modes*. The required area (A_{ra}), in frames, for a PR region is the net area required for implementing all the modes assigned to it. This area is calculated by taking the maximal resource requirement for each resource type, in tiles, and multiplying by the number of frames needed for each tile of that type: Note that there is some overhead in this resource requirement due to it being based on whole tiles.

$$A_{ra} = \sum_{j} W_j * N_j, \qquad j \; \epsilon \; CLB, DSP, BlockRAM. \tag{4.1}$$

where W_j is the number of *frames* per type of tile j and N_j is the number of tiles of type j needed.

4.3.4 Actual Reconfigurable Area

When a design is placed, the actual area may differ from the initial requirement due to the rectangular shape requirement for PR regions or the disparate arrangement of resources on the FPGA fabric. Mathematically, the actual area (A_{aa}) of a region is calculated as

$$A_{aa} = \sum_{i} W_i * N_i, \qquad i \; \epsilon \; CLB, DSP, BlockRAM. \tag{4.2}$$

where W_i is the number of *frames* per tile of type i and N_i is the number of tiles of type i allocated in the region. The result is the number of frames to configure the placed region.

4.3.5 Resource Wastage

The resource wastage for a particular placement of a reconfigurable region (A_{rw}) is the difference between the actual area and the required area of that region, in frames.

The total resource wastage of a full floorplan (A_{tw}) is the sum of resource wastage among all the regions.

$$A_{rw} = A_{aa} - A_{ra}. \tag{4.3}$$

$$A_{tw} = \sum_{r} A_{rw}. \tag{4.4}$$

While floorplanning partial regions, the floorplanner should try to minimise the total resource wastage in order to minimise reconfiguration time and maximise the resources available for implementing static logic.

4.3.6 Wirelength

Total wirelength is an important parameter in determining the effectiveness of floorplanning. Here we consider the Manhattan distance between regions and the total wirelength between two regions is calculated as the product of the Manhattan distance between them and the number of wires connecting them. Several previous researches consider total Half Perimeter Wire Length (HPWL) as the minimisation objective function for their floorplanner. Practically, HPWL has very little impact in FPGA floorplanning. In ASIC floorplanning, HPWL gives a figure of compactness of cells and hence the best timing achievable, but in FPGAs, where all resources as well as routing between them are fixed, HPWL does not give an accurate measure of timing performance.

4.3.7 Static Logic

Static logic is the area of the FPGA with fixed functionality. I/O pins are always assigned to the static region, since assigning I/O pins to reconfigurable regions may cause undesirable switching during reconfiguration. There is no restriction on the shape of static logic. To make optimal use of FPGA resources, and achieve timing closure, it is better not to restrict the shape of static logic or allocate a special location for it. The reconfigurable regions should be floorplanned in such a way that, the area available for the implementation of static logic is maximised, and it should

be floorplanned after the PR regions.

4.4 Proposed Floorplanner

The input to our proposed floorplanner is the partition information of reconfigurable regions and their connectivity information. A connectivity matrix is used, whose element (i, j) represents the number of nets between region i and region j. The output of the floorplanner is a set of area constraints, which specify the coordinates of the bottom left and top right corners of each region. These constraints can be used for generating the *user constraints* file, which will be used by the vendor specific place and route tool for generating the final configuration bitstreams. The floorplanning problem can be formulated as follows:

Given:

- M regions with resource requirement 3-tuple, (n_{CLB}, n_{BR} and n_{DSP}) for each region,
- an FPGA of width W and height H,
- with N_{CLB}, N_{BR} and N_{DSP} resources available,
- and R device rows,

partition the FPGA into M rectangles, so that:

- each region can be mapped into a rectangle, which contains sufficient resources,
- rectangle height being an integer multiple of device row,
- no rectangles overlapping,
- minimising the cost function.

The outputs are the (x_{min},y_{min}) and (x_{max},y_{max}) coordinates of each rectangle so that $0 \leq x_{min} \leq x_{max} \leq W$ and $0 \leq y_{min} \leq y_{max} \leq H$.

4.4.1 Columnar Kernel Tessellation

Mapping an area directly using FPGA primitives is not practical, due to number of factors such as the large search space, limited number of available primitives in the FPGA, fixed primitive locations, rectangular shape region constraint, etc. Hence we adopt a new method called Columnar Kernel Tessellation. A kernel is a structure one device row high, containing FPGA primitives, which can be repeated in the vertical direction to satisfy a region's resource requirements. The availability of kernels for floorplanning a region changes based on the floorplanning of previous regions. The smallest kernel is a single tile. Each tile can be clustered with nearby tiles and can form new kernels.

The first step of floorplanning is to calculate the resource usage of each region in terms of reconfigurable tiles. For this purpose, the input resource utilisation values are divided by the corresponding number of resources available in a tile. This may result in some overhead if the resources needed do not use a whole tile. For example in Virtex-5 FPGAs, the required number of CLBs will be divided by 20, DSPs by 8, and Block RAMs by 4. The floorplanner maintains a database of FPGA architectures that contains information about the resource type of each device column. The different types of columns are mapped to a single co-ordinate system for better management. Each tile in the FPGA is encoded using a data-structure with information including location, resource type, used or not, and availability. Once a tile is used to floorplan a region, its *use field* is set to true. The tiles belonging to the locations of hard processors and transceivers are set to be unavailable.

In order to perform floorplanning, the regions are initially sorted according to descending resource requirements, into a floorplanning schedule. Regions are selected based on the following ordered criteria.

1. Require both DSP as well as Block RAM tiles,
2. Require DSP and CLB tiles,
3. Require Block RAM and CLB tiles,
4. Require CLBs tiles alone.

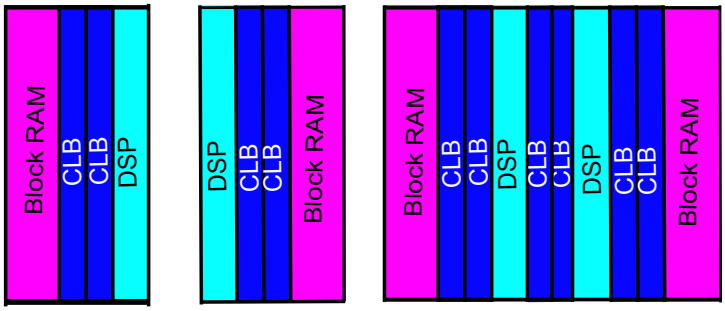


Figure 4.1: Two Block RAM-DSP kernels and a merged kernel.

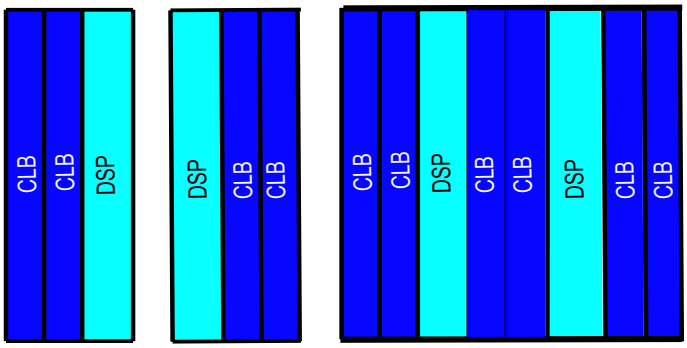


Figure 4.2: Two Block DSP-CLB kernels and a merged kernel.

This classification is based on the fact that DSP tiles are the least available and hence the most precious FPGA resource. Block RAM tiles are weighted next and CLB tiles are the most abundant resource available, and so given the least weighting. Regions belonging to each group are sorted in descending order of DSP, Block RAM and CLB tiles required. Regions are selected from the scheduling list in sequential order and packed.

The floorplanner starts with regions which use both DSP tiles and Block RAM tiles. The floorplanner selects a kernel, which contains both DSP and Block RAM tiles. To generate kernels, the resource column information from the database is utilised. For each DSP column, the nearest Block RAM column location is calculated. The nearest tiles of DSP and Block RAM along with the tiles between them are merged to create kernels. These kernels are merged again and larger kernels are created. This operation is illustrated in Fig. 4.1 in the case of a Virtex-5 FX70T FPGA. It is to note that when kernels are merged, the CLB tiles in between them are

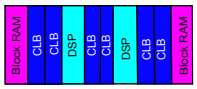
Kernel	#DSP tiles	#BR tiles	#CLB tiles	#Frames	#Bytes
	1	1	2	28 + 30 + 36 = 94	94 * 164 = 15416
	2	2	6	2*28 + 2*30 + 6*36 = 332	332 * 164 = 54448

Figure 4.3: Example calculation of the size of kernel.

also included in the resulting kernel. All kernels are one device row high. From the set of available kernels, the kernel with smallest size is chosen and used for packing. Example calculation of kernel size is shown in Fig. 4.3. Kernels are repeated in a columnar direction to meet the region's resource requirements. The minimum number of kernels required for packing is equal to the number of DSP tiles required divided by the number of DSP tiles in the kernel. The maximum number of kernels that can be used to satisfy the DSP resource requirement is equal to the number of device rows, i.e. the full device height, since the height of a kernel is one device row. If the arrangement of a kernel cannot meet the required number of DSP tiles, that kernel is discarded and the kernel with next lowest resource requirement is selected and used for packing.

Once the DSP-BR kernels are packed, the remaining BR and CLB resources required for that region are calculated. If more Block RAMs are needed, the nearest Block RAM column is selected from the database and used. Preference is given to columns towards the right and left edges of the FPGA in order to maximise free space available towards the centre of the FPGA. If more CLB tiles need to be allocated, CLB columns towards the device edge are selected and allocated. Once the allocation is performed, the tiles which are used are marked in the database as *used*.

Now the regions which use only DSP and CLB tiles are packed. For this purpose, the kernels considered contain only DSP tiles and CLB tiles. The minimum number of kernels required for packing will be equal to the number of DSP tiles required divided by the number of DSP tiles in the kernel. Tiles which are not marked as *used*

or *not available* are used to generate the new set of kernels. This same process is followed once more for regions containing Block RAMs. Finally, regions containing only CLB tiles are packed.

The inherent rectangular shape of kernels and the columnar repetition guarantees that the allocated area for each region will be of rectangular shape and region height will be an integer multiple of device rows. The floorplanner follows a divide and conquer method. The packing of each region reduces the search space for implementing subsequent regions as well as the number of kernels available. The algorithm runs a number of times, each time starting with a different kernel for packing. The number of iterations can be specified or can be stopped when a required cost objective is met. At the end of each complete packing, a cost function is evaluated for the floorplan . The cost function is defined as

$$CF = \alpha * A_{tw} + \beta * WL. \tag{4.5}$$

where A_{tw} is the total resource wastage and WL is the total wirelength between regions. α and β are weight factors with $\alpha > \beta$. For designs where reconfiguration time is highly critical compared to speed of operation, the value of β can be set to zero and for applications where timing is highly critical rather than reconfiguration time α can be set to zero. For other applications, the value of α and β are weighted accordingly.

At the end of each complete floorplan generation, a post processing step is performed, in which the regions are moved along the columnar direction towards the middle of the columns. If this movement improves wirelength, the movement is accepted otherwise it is rejected. Also, regions that occupy the same columns are swapped and wirelength is recalculated. This move is also accepted only if it improves wirelength. These moves are a type of pseudo-simulated-annealing. This is possible due to the fact that the resources are arranged in columnar fashion in the FPGA, and moving a region along its columns does not affect resource availability, provided there are no unavailable tiles in the direction of movement.

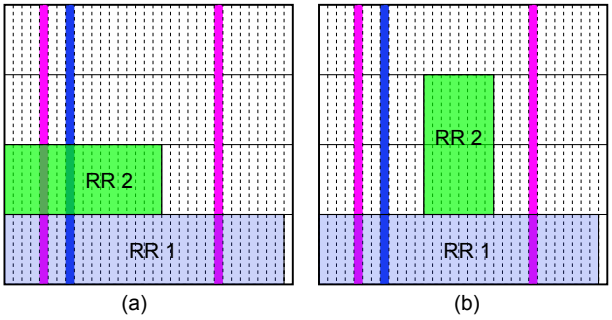


Figure 4.4: (a) Resulting floorplan from [1], (b) Resulting floorplan from our method.

4.5 Case Study

A direct comparison of our method with existing methods is not possible due to the unavailability of other floorplanning tools and uniform benchmark circuits. Hence, we use a reported case study, taken from [1], and compared the results using our method. The system implemented consists of a CAN controller, Floating Point Unit (FPU), FIR filter, CRC controller and an Ethernet controller. Based on the partitioning algorithm, modules are partitioned into two reconfigurable regions. The CAN controller, FPU and CRC are implemented in reconfigurable region 1 (RR 1) and FIR filter and Ethernet controller are implemented in region 2 (RR2), as per [1]. The design is implemented in a Virtex-5 LX30T device. Region 1 requires 24% of the available CLB and 5 Block RAMs and region 2 requires 13% of CLBs. The static region requires 61% of CLBs and 40 Block RAMs. The resulting floorplan reported in the paper is given in Fig. 4.4.a. It is clear that although region 2 does not require Block RAMs or DSP slices, the resulting floorplan includes these resources. This leads to increased region size, higher configuration time and additional storage requirement. Furthermore, these resources cannot be used elsewhere in the design. This floorplan uses a total of 1766 frames.

A floorplan determined by our method is shown in Fig. 4.4.b. Region 2 is floorplanned in such a way that no DSP slices and Block RAMs are used. Hence our method uses 58 fewer frames and reserves more resources for static logic implementation. The smaller size of the region also contributes to 18.4 KB (9.2×2 since there are two partial bitstreams for that region) less storage requirement and a corresponding

improvement in reconfiguration time.

For a more complex investigation, we could find no existing work to compare to, nor standard tools to use, so we floorplanned an in-house design using our proposed method and compared it to an ad-hoc floorplan based on previous experience with some optimisation effort. The selected design is a software defined radio (SDR) targeted for Xilinx Virtex-5 FX70T FPGA. The SDR chain consists of a matched filter, carrier recovery circuit, demodulator, signal decoder and video decoder. Each module has a number of *modes* with different resource requirements. *Modes* are mutually exclusive implementations of the module with the same set of inputs and outputs. Here we assume each module is assigned to a single region, and hence the resource requirement of each region is the requirement of the largest *mode* of the module assigned to it. Here, all modules are connected in sequential order with a 64 bit wide bus. The static logic contains a PowerPC-440 embedded processor, external memory interface and an ICAP controller. The different regions and associated resource requirements are given in Table 4.1. The $rq'd$ field indicates the exact number of resources required, the *tiles* field indicates the required number of tiles needed to satisfy the resource requirement and the *waste* field indicates the resources wasted due to rounding the resources to tiles. The total number of frames wasted due to the tiling operation is roughly 115 frames.

Table 4.1: Resource utilisation for reconfigurable regions.

Region	CLBs			BRs			DSPs			#Frms
	Rq'd	Tiles	Wst	Rq'd	Tiles	Wst	Rq'd	Tiles	Wst	
Matched Filt.	500	25	0	0	0	0	34	5	6	1040
Carrier Rec.	123	7	17	0	0	0	8	1	0	280
Demodulator	97	5	3	8	2	0	0	0	0	240
Decoder	234	12	6	2	1	2	0	0	0	462
Video Dec.	1100	55	0	6	2	2	34	5	6	2180
Total	2054	104	26	16	5	4	76	11	12	4202

The ad-hoc floorplanning is done as per Xilinx PR floorplanning guidelines, with the help of the PlanAhead software. Modules are selected one by one in the order they appear in the processing chain and placed using the PlanAhead GUI. Rectangular

regions are chosen with the help of the software, which shows the amount of resources present in the selected region. Modules were tried to place as close as possible for getting better performance.

While using our algorithm, as per the description in Section 4.1, the order for floorplanning regions is the video decoder first, followed by the matched filter, carrier recovery, demodulator and finally the decoder. The result of the ad-hoc floorplanning and 5 of the best floorplans using our method are given in Table 4.2.

Table 4.2: Resource wastage and total wirelength for different floorplans.

Plan No.	Wastage, A_{tw} (frames)	Wirelength, WL (Normalised)
Adhoc	956	7420
Plan1	466	8640
Plan2	486	9056
Plan3	592	11776
Plan4	516	7392
Plan5	556	9120

There is no fixed relationship between the resource wastage and wirelength, owing to the rectangular shape requirement of the reconfigurable regions as well as the disparate arrangement of resources. The ad-hoc plan, the plan which produces minimum resource wastage (Plan1) and the plan which gives minimum wirelength (Plan4) are shown in Fig. 4.5. When the value of α is set to zero in the cost function, Plan 4 is the preferred floorplan, and when β is set to zero, Plan 1 gives the best results. We can see that the proposed floorplanner performs well on area: all the floorplans have lower resource wastage from 38% to 51%, which corresponds to a decrease in reconfiguration from 7% to 9.5% compared to the ad-hoc plan. Since the modules of the regions are in a continuous chain, the ad-hoc method is able to achieve good total wirelength. In usual FPGA floorplanning without PR, the ad-hoc floorplan is fairly acceptable, since all the resource requirements are satisfied and wirelength is minimised. But for PR designs, the resource wastage creates a considerable overhead in terms of reconfiguration time. Moreover, storing 956 frames of configuration frames requires 153 KB extra storage memory for each system configuration. Other

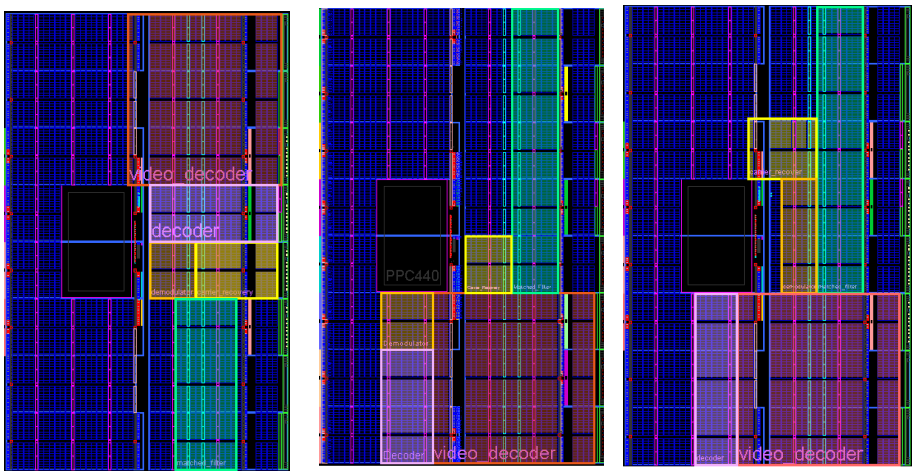


Figure 4.5: Floorplans using (a) An ad-hoc approach, (b) proposed method with minimum resource wastage, (c) with minimum wirelength.

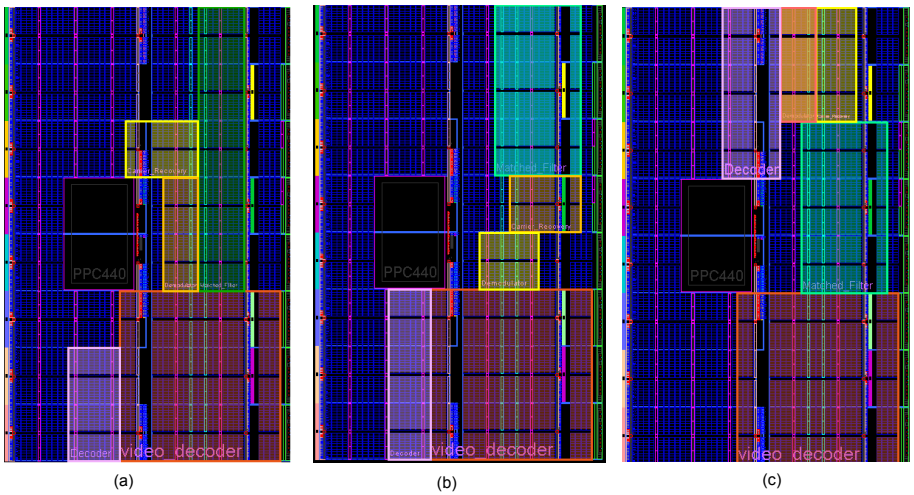


Figure 4.6: Suboptimal Floorplans (a) Plan-2, (b) Plan-3, (c) Plan-5.

floorplans are shown in Fig. 4.6.

These results demonstrate the advantage of considering the required implementation metrics in floorplanning. While static designs only require the floorplan to fit and achieve timing, a PR design's reconfiguration time is also affected by the floorplan. Our approach ensures this is factored into the floorplanning process, and results in savings as shown.

4.6 Conclusion

In this section we introduced a novel method for PR design floorplaning. The method described is fully compatible with the latest vendor-supported PR tool-flows. The heterogeneous and irregular architecture of modern FPGA families are considered and floorplanning cost functions tailored for PR are introduced. It is worth noting that, this floorplanning method is also compatible with older generations of FPGAs such as Virtex-2. In older generations of FPGAs, configuration frames extended the whole height of the device, instead of a single device row. Our study proves that it is possible to optimise the area requirement considering the tile constraint. We have also found that a significant area overhead is generated due to the tiling requirement of reconfigurable regions. Hence, considering tiling at the partitioning stage, prior to floorplanning may yield more efficient designs.

4.7 Related Publication

The work in this chapter was published in:

1. Kizheppatt Vipin, Suhaib A Fahmy, *Architecture-Aware Reconfiguration-Centric Floorplanning for Partial Reconfiguration*, Proc. of International Symposium on Applied Reconfigurable Computing, Hong Kong, 2012.

Chapter 5

Future Research and Conclusion

Through this research work, we aim to develop a framework for adaptive system design using partial reconfiguration. The framework can be broadly classified into two parts, design-time PR methods and tools and run-time PR support. Design time methods and tools aim at providing support to the designers during the hardware design stage of the system. Specifically speaking, we aim to automate several operations the designers currently have to perform manually, while optimising the efficiency of the system. Development of automatic partitioning and floorplanning comes under this category. Our immediate priority will be the integration of the partitioning tool with the floorplanner. Once it is done, a number of real world systems will be tested for the efficiency of the proposed algorithms. An XML based framework is being developed, which will enable the integration of these tools with the vendor provided synthesis and place and route tools. The designer can provide a high level system specification in an XML file including the module and mode names, their corresponding HDL files and timing constraints. A Python programme will parse the XML file and extract the required information. Vendor provided tools will be automatically invoked along with the custom developed tools. The result will be an efficiently-planned PR design, along with all the necessary bitstreams.

The design-time contributions detailed above provide everything necessary for the PR system to be implemented. However, true adaptive systems need intelligent management of adaptation that is typically better done in software. During the design

stage, the proposed tools build necessary infrastructure and interfaces for the management of PR. During runtime the system adapts based on operational conditions. An adaptive system can be considered as having two planes. The data plane implements the processing of data, such as the signal processing in a radio. The system designer uses a high level tool to describe this based on a library of IP cores. The control plane implements the management and control functionalities. This plane is implemented in software due to its flexibility.

Adaptations are managed by the control plane by determining the appropriate type of configuration and loading the corresponding bitstreams or configuring internal register values. Current PR tools do not provide automated support for different levels of reconfiguration. Adaptation needs to be explicitly coded by the designer, who must explicitly state which bitstreams to load. A design framework needs to be developed so that designers with less hardware expertise can specify complex adaptations using a high level system specification. From this high level description, the runtime should be able to enact the appropriate reconfiguration. The levels of reconfiguration are abstracted away and should be transparent to the system designer, with the mapping of adaptation to physical reconfiguration automated.

Also we intend to develop a library of IP cores targeting adaptive systems development. The cores will be targeting software defined radio and automotive domains. Designers will be able to easily integrate these cores into their systems. We are currently doing studies on bus architectures, which will enable seamless integration of these cores. Adoption of AXI interface by Xilinx for their new IP cores is encouraging though we are also looking at the open-source Wishbone bus.

Currently we have identified the key challenging areas and objectives. An efficient partitioning algorithm and a floorplanner are already complete. As the next step, we are integrating the partitioning tool with the floorplanner and vendor place and route tools. Specific domains are identified, for which library components need to be developed. We are now working on the program that will automate these design-time steps. We believe that the availability of better tools and support will lead to wider adoption of PR in adaptive systems, for which we believe FPGAs offer the ideal mix of higher performance and flexibility.

A planned timeline for our future research direction is given in Fig. 5.1. We aim to publish the research outcomes in prestigious conferences such as International Conference on Field Programmable Logic and Applications-2012 (FPL-2012), ACM/SIGDA International Symposium on Field-Programmable Gate Arrays-2013 (FPGA-2013), International Conference on Hardware/Software Codesign and System Synthesis-2013 (CODES+ISSS -2013), and journals including IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD), ACM Transactions on Reconfigurable Technology and Systems (TRETS) and IEEE Transactions on Communications (TCOM).

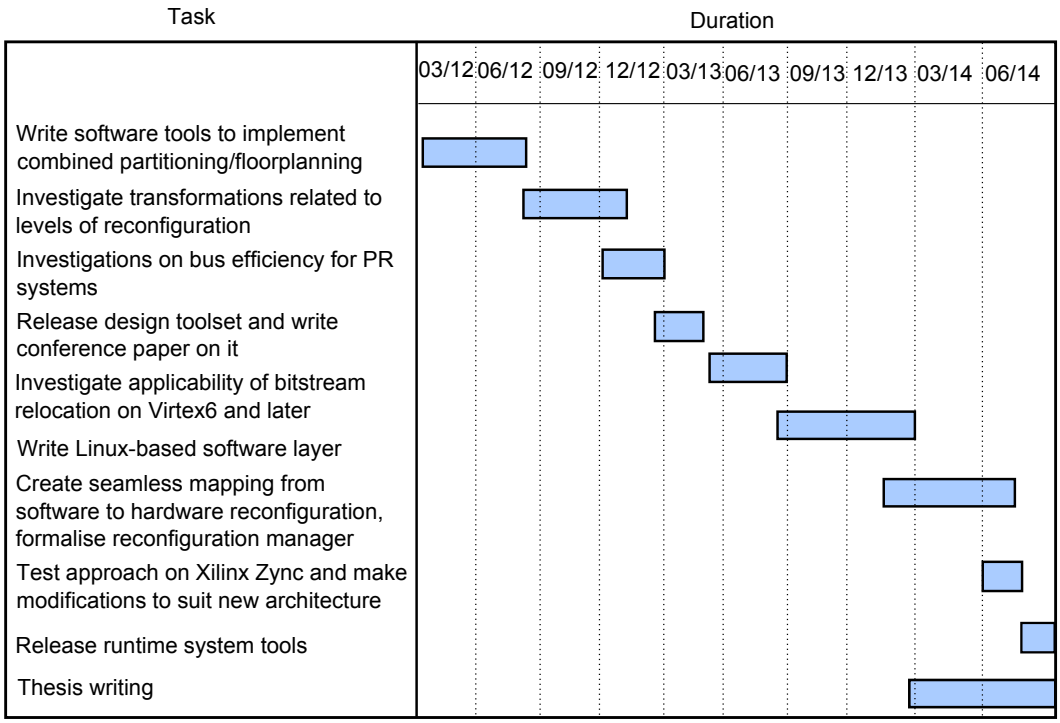


Figure 5.1: Future Research Directions.

Publications

The work completed so far has been written up in a number of published and submitted papers

1. K. Vipin, S.A. Fahmy, *Efficient Region Allocation for Adaptive Partial Reconfiguration*, Proc. of IEEE International Conference on Field Programmable Technology (FPT), New Delhi, 2011.

2. K. Vipin, S.A. Fahmy, *Enabling High Level Design of Adaptive Systems with Partial Reconfiguration*, Proc. of IEEE International Conference on Field Programmable Technology (FPT), New Delhi, 2011.

3. K. Vipin, S.A. Fahmy, *Architecture-Aware Reconfiguration-Centric Floorplanning for Partial Reconfiguration*, to appear in Proc. of International Symposium on Applied Reconfigurable Computing, Hong Kong, 2012.

Currently we are preparing two manuscripts for publication

1. K. Vipin, S.A. Fahmy, *A Survey on FPGA Partial and Dynamic Reconfiguration: Architectures, Tools, and Applications.*

2. K. Vipin, S.A. Fahmy, *An Automated Partitioning Scheme for Partial Reconfiguration based Adaptive Systems.*

Bibliography

[1] A. Montone, M.D. Santambrogio, D. Sciuto, and S.O. Memik. Placement and floorplanning in dynamically reconfigurable FPGAs. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, 3(4):24:11–24:34, Nov. 2010.

[2] C. Claus, J. Zeppenfeld, F. Muller, and W. Stechele. Using partial-run-time reconfigurable hardware to accelerate video processing in driver assistance system. In *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2007.

[3] J. Lotze, S.A. Fahmy, J. Noguera, B. Ozgul, L. Doyle, and R. Esser. Development framework for implementing FPGA-based cognitive network nodes. In *Proceedings of IEEE Global Telecommunications Conference (GLOBECOM)*, 2009.

[4] J.P. Delahaye, J. Palicot, C. Moy, and P. Leray. Partial reconfiguration of FPGAs for dynamical reconfiguration of a software radio platform. In *Proceedings of IST Mobile and Wireless Comms. Summit*, 2007.

[5] J. Diaz, E. Ros, F. Pelayo, E.M. Ortigosa, and S. Mota. FPGA-based real-time optical-flow system. *IEEE Transactions on Circuits and Systems for Video Technology*, 16(2):274–279, 2006.

[6] S. Liu, R.N. Pittman, A. Forin, and J.L. Gaudiot. On energy efficiency of reconfigurable systems with run-time partial reconfiguration. In *Proceedings of IEEE International Conference on Application-specific Systems Architectures and Processors (ASAP)*, 2010.

[7] J. Becker, A. Donlin, and M. Huebner. New tool support and architectures in adaptive reconfigurable computing. In *Proceedings of IFIP International Conference on Very Large Scale Integration (VLSI - SoC)*, 2007.

[8] Actel. *ProASIC3 Flash FPGAs*, 2010.

[9] M. Peattie. Using a microprocessor to configure Xilinx FPGAs via slave serial or SelectMAP mode. Technical report, Xilinx Inc., 2009.

[10] Xilinx Inc. *DS586: LogiCORE IP XPS HWICAP*, 2010.

[11] W. Chong, S. Ogata, M. Hariyama, and M. Kameyama. Architecture of a multi-context FPGA using reconfigurable context memory. In *Proceedings of IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2005.

[12] R.T. Ong. Programmable logic device which stores more than one configuration and means for switching configurations., Jun 1995.

[13] S. Trimberger, D. Carberry, A. Johnson, and J. Wong. A time-multiplexed FPGA. In *Proceedings of IEEE Symposium on FPGAs for Custom Computing Machines (FCCM)*, 1997.

[14] I. Kennedy. Exploiting redundancy to speedup reconfiguration of an FPGA. In *In Proceedings of International Conference on Field Programmable Logic and Applications (FPL)*, 2003.

[15] S.M. Scalera and J.R. Vazquez. The design and implementation of a context switching FPGA. In *Proceedings of IEEE Symposium on FPGAs for Custom Computing Machines (FCCM)*, 1998.

[16] K Puttegowda, D.I. Lehn, J.H. Park, P. Athanas, and M. Jones. Context switching in a run-time reconfigurable system. *The Journal of Supercomputing*, 26(3):239–257, 2003.

[17] Xilinx Inc. *Programmable Logic Data Book*, 1996.

[18] Xilinx Inc. *DS031:Virtex-II Platform FPGAs*, 2003.

[19] Xilinx Inc. *DS083: Virtex-II Pro and Virtex-II Pro-X Platform FPGAs*, 2011.

[20] Xilinx Inc. *XAPP151: Virtex Series Configuration Architecture User Guide*, 2004.

[21] Xilinx Inc. *UG070: Virtex-4 FPGA User Guide*, 2008.

[22] P. Lysaght, B. Blodget, J. Mason, J. Young, and B. Bridgford. Invited paper: Enhanced architectures, design methodologies and CAD tools for dynamic reconfiguration of xilinx FPGAs. In *Proceedings of International Conference on Field Programmable Logic and Applications (FPL)*, 2006.

[23] Xilinx Inc. *UG191: Virtex-5 FPGA Configuration User Guide*, 2010.

[24] Altera. Uf-01002: Upcoming Stratix-V device features. Technical report, 2011.

[25] Altera. *SIV52005: Dynamic Reconfiguration in Stratix IV Devices*, 2005.

[26] Lattice Semiconductor Corporation. *ORCA Series 4 FPGAs*, March 2003.

[27] Atmel. AT40K series configuration. Technical report, Atmel, 2002.

[28] Tabula. Spacetime architecture. Technical report, Tabula, 2010.

[29] C. Dong, D. Chen, S. Haruehanroengra, and W. Wang. 3-D nFPGA: a reconfigurable architecture for 3-D CMOS/Nanomaterial hybrid digital circuits. *IEEE Transactions on Circuits and Systems (TCAS)*, 54(11):2489–2501, Nov 2007.

[30] R. David, D. Chillet, S. Pillement, and O. Sentieys. DART: a dynamically reconfigurable architecture dealing with future mobile telecommunications constraints. In *Proceedings of International Parallel and Distributed Processing Symposium (IPDPS)*, 2002.

[31] S. Saito, Y. Kohama, Y. Sugimori, Y. Hasegawa, H. Matsutani, T. Sano, K. Kasuga, Y. Yoshida, K. Niitsu, N. Miura, T. Kuroda, and H. Amano. MuCCRA-Cube: A 3D dynamically reconfigurable processor with inductive-coupling link.

In *Proceedings of International Conference on Field Programmable Logic and Applications (FPL)*, 2009.

[32] D. Alnajiar, Y. Ko, T. Imagawa, H. Konoura, M. Hiromoto, Y. Mitsuyama, M. Hashimoto, H. Ochi, and T. Onoye. Coarse-grained dynamically reconfigurable architecture with flexible reliability. In *Proceedings of International Conference on Field Programmable Logic and Applications (FPL)*, 2009.

[33] E. Eto. Xapp290: Difference-based partial reconfiguration. Technical report, Xilinx Inc., 2007.

[34] J. Harkin, T.M. Mcginnity, and L.P. Maguire. Modeling and optimizing run-time reconfiguration using evolutionary computation. *ACM Transactions on Embedded Computing Systems (TECS)*, 3(4):661–685, Nov. 2004.

[35] W. Luk, N. Shirazi, and P.Y.K. Cheung. Modelling and optimising run-time reconfigurable systems. In *Proceedings of IEEE Symposium on FPGAs for Custom Computing Machines (FCCM)*, 1996.

[36] C. Sorel and Y. Lavarenne. From algorithm and architecture specifications to automatic generation of distributed real-time executives: A seamless flow of graphs transformations. In *Proceedings of Formal Methods and Models for Codesign Conference*, 2003.

[37] F. Berthelot, F. Nouvel, and D. Houzet. Partial and dynamic reconfiguration of FPGAs: a top down design methodology for an automatic implementation. In *International Symposium on Parallel and Distributed Processing (IPDPS)*, 2006.

[38] I. Robertson and J. Irvine. A design flow for partially reconfigurable hardware. *ACM Transactions on Embedded Computing Systems (TECS)*, 3(2):257–283, May 2004.

[39] M. Boden, T. Fiebig, M. Reiband, and P. Reichel. Gepard - a high-level generation flow for partially reconfigurable designs. In *Proceedings of IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2008.

[40] N. Abel. Design and implementation of an object-oriented framework for dynamic partial reconfiguration. In *International Conference on Field Programmable Logic and Applications (FPL)*, 2010.

[41] S.A. Fahmy, J. Lotze, J. Noguera, L. Doyle, and R. Esser. Generic software framework for adaptive applications on FPGAs. In *Proceedings of IEEE Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2009.

[42] H. Tan and R.F. DeMara. A multilayer framework supporting autonomous runtime partial reconfiguration. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 16(5):504–516, May 2008.

[43] C. Conger, R. Hymel, M. Rewak, A.D. George, and H. Lam. FPGA design framework for dynamic partial reconfiguration. In *Proceedings of Reconfigurable Architectures Workshop (RAW)*, 2008.

[44] L. Singhal and E. Bozorgzadeh. Multi-layer floorplanning on a sequence of reconfigurable designs. In *Proceedings of International Conference on Field Programmable Logic and Applications (FPL)*, 2006.

[45] P. Banerjee, M.Sangtani, and S.Sur-Kolay. Floorplanning for partial reconfiguration in FPGAs. In *Proceedings of International Conference on VLSI Design*, 2009.

[46] K. Vipin and S.A. Fahmy. Efficient region allocation for adaptive partial reconfiguration. In *Proceedings of the International Conference on Field Programmable Technology (FPT)*, 2011.

[47] S.N. Adya and I.L. Markov. Fixed-outline floorplanning through better local search. In *Proceedings of ACM/IEEE International Conference on Computer Design*, 2001.

[48] Y. Feng and D.P. Mehta. Heterogeneous floorplanning for FPGAs. In *Proceedings of International Conference on VLSI Design*, 2006.

[49] J. Yuan, S. Dong, X. Hong, and Y. Wu. LFF algorithm for heterogeneous FPGA floorplanning. In *Proceedings of Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2005.

[50] P. Banerjee, M. Sangtani, and S. Sur-Kolay. Floorplanning for partially reconfigurable FPGAs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 30(1):8–17, Jan. 2011.

[51] P. Yuh, C. Yang, and Y. Chang. Temporal floorplanning using the T-tree formulation. In *Proceedings of IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2004.

[52] P. Yuh, C. Yang, Y. Chang, and H. Chen. Temporal floorplanning using 3D-subTCG. In *Proceedings of Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2004.

[53] L. Singhal and E. Bozorgzadeh. Multi-layer floorplanning for reconfigurable designs. *IET Computers & Digital Techniques*, 1(4):276–294, July 2007.

[54] Y. Zhan, Y. Feng, and S. Sapatnekar. A fixed-die floorplanning algorithm using an analytical approach. In *Proceedings of Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2006.

[55] K. Bazargan, R. Kastner, and M. Sarrafzadeh. Fast template placement for reconfigurable computing systems. *IEEE Design and Test of Computers*, 17(1):68–83, Jan 2000.

[56] Y. Lu, T. Marconi, G.N. Gaydadjiev, K. Bertels, and R.J. Meeuws in RAW2008. A self-adaptive on-line task placement algorithm for partially reconfigurable systems. In *Proceedings of Parallel and Distributed Processing Symposium (IPDPS)*, 2008.

[57] S. Raaijmakers and S. Wong. Run-time partial reconfiguration for removal, placement and routing on the Virtex-II Pro. In *International Conference on Field Programmable Logic and Applications (FPL)*, 2007.

[58] S.P. Fekete, J.C. van der Veen, A. Ahmadinia, D. Ghringer, M. Majer, and J. Teich. Offline and online aspects of defragmenting the module layout of a partially reconfigurable device. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 16(9):1210–1219, Sept. 2008.

[59] K. Compton, Z. Li, J. Cooley, and S. Knol. Configuration relocation and defragmentation for run-time reconfigurable computing. *IEEE Transactions on VLSI Systems*, 10(3):209–220, 2002.

[60] M. Koester, W. Luk, J. Hagemeyer, and M. Porrmann. Design optimizations to improve placeability of partial reconfiguration modules. In *Proceedings of Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2009.

[61] T. Becker, W. Luk, and P.Y.K. Cheung. Enhancing relocatability of partial bitstreams for run-time reconfiguration. In *Proceedings of IEEE Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2007.

[62] H. Kalte, G. Lee, M. Porrmann, and U. Rckert. REPLICA: A bitstream manipulation filter for module relocation in partial reconfigurable systems. In *Proceedings of IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2005.

[63] H. Kalte and M. Porrmann. REPLICA2Pro: Task relocation by bitstream manipulation in Virtex-II/Pro FPGAs. In *Proceedings of conference on Computing frontiers*, 2006.

[64] S. Guccione, D. Levi, and P. Sundararajan. JBits: Java based interface for reconfigurable computing. Technical report, Xilinx Inc., 2004.

[65] A. Ahmadinia, C. Bobda, J. Ding, M. Majer, J. Teich, S.P. Fekete, and J.C van der Veen. A practical approach for circuit routing on dynamic reconfigurable devices. In *Proceedings of IEEE International Workshop on Rapid System Prototyping (RSP)*, 2005.

[66] P. Athanas, J. Bowen, T. Dunham, C. Patterson, J. Rice, M. Shelburne, J. Suris, M. Bucciero, and J. Graf. Wires on demand: Run-time communication synthesis for reconfigurable computing. In *Proceedings of International Conference on Field Programmable Logic and Applications (FPL)*, 2007.

[67] D. Koch, C. Haubelt, and J. Teich. Efficient reconfigurable On-Chip buses for FPGAs. In *Proceedings of International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2008.

[68] D. Koch, C. Beckhoff, and J. Teich. ReCoBus-Builder a novel tool and technique to build statically and dynamically reconfigurable systems for FPGAs. In *Proceedings of International Conference on Field Programmable Logic and Applications (FPL)*, 2008.

[69] A. Jara-Berrocal and A. Gordon-Ross. SCORES: A scalable and parametric streams-based communication architecture for modular reconfigurable systems. In *Proceedings of Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2009.

[70] S. Liu, R. N. Pittman, A. Forin, and J. Gaudiot. On energy efficiency of reconfigurable systems with run-time partial reconfiguration. In *IEEE International Conference on Application-specific Systems Architectures and Processors (ASAP)*, 2010.

[71] C. Claus, F. H. Muller, J. Zeppenfeld, and W. Stechele. A new framework to accelerate Virtex-II Pro dynamic partial self-reconfiguration. In *Proceedings of IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2007.

[72] S. G. Hansen, D. Koch, and J. Torresen. High speed partial run-time reconfiguration using enhanced ICAP hard macro. In *Proceedings of IEEE International Symposium on Parallel and Distributed Processing Workshops and Phd Forum (IPDPSW)*, 2011.

[73] J. H. Pan, T. Mitra, and W. Wong. Configuration bitstream compression for dynamically reconfigurable FPGAs . In *Proceedings of IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2004.

[74] S. Hauck, Z. Li, and E. Schwabe. Configuration compression for the xilinx XC6200 FPGA. In *Proceedings of IEEE Symposium on FPGAs for Custom Computing Machines (FCCM)*, 1998.

[75] S. Hauck, Z. Li, and E. Schwabe. Configuration compression for the xilinx XC6200 FPGA. In *Proceedings of IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 1999.

[76] Z. Li and S. Hauck. Configuration compression for Virtex FPGAs. In *Proceedings of IEEE Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2001.

[77] G. Haiyun and C. Shurong. Partial reconfiguration bitstream compression for Virtex FPGAs. In *Congress on Image and Signal Processing (CISP)*, 2008.

[78] B. Sellers, J. Heiner, M. Wirthlin, and J. Kalb. Bitstream compression through frame removal and partial reconfiguration. In *Proceedings of International Conference on Field Programmable Logic and Applications (FPL)*, 2009.

[79] S. Ganesan and R. Vemuri. An integrated temporal partitioning and partial reconfiguration technique for design latency improvement. In *Proceedings of Design, Automation and Test in Europe Conference and Exhibition (DATE)*, 2000.

[80] Z. Li and S. Hauck. Configuration prefetching techniques for partial reconfigurable coprocessor with relocation and defragmentation. In *Proceedings of ACM/SIGDA International Symposium on Field-programmable Gate Arrays (FPGA)*, 2002.

[81] V. Rana, S. Murali, D. Atienza, M. D. Santambrogio, L. Benini, and D. Sciuto. Minimization of the reconfiguration latency for the mapping of applications on

FPGA-based systems. In *Proceedings of IEEE/ACM International Conference on Hardware/software Codesign and System Synthesis (CODES+ISSS)*, 2009.

[82] Z. Li, K. Compton, and S. Hauck. Configuration caching management techniques for reconfigurable computing. In *Proceedings of IEEE Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2000.

[83] S. Ghiasi and M. Sarrafzadeh. Optimal reconfiguration sequence management. In *Proceedings of Asia and Sourth Pacific Design Automation Conference (ASP-DAC)*, 2003.

[84] S. Ghiasi, A. Nahapetian, and M. Sarrafzadeh. An optimal algorithm for minimizing run-time reconfiguration delay. *ACM Transactions on Embedded Computing Systems (TECS)*, 3(2):237–256, May 2004.

[85] W. Lie and W.Feng-yan. Dynamic partial reconfiguration on cognitive radio platform. In *Proceedings of IEEE International Conference on Intelligent Computing and Intelligent Systems (ICIS)*, 2009.

[86] C.S. Choi and H. Lee. An reconfigurable fir filter design on a partial reconfiguration platform. In *Proceedings of Communications and Electronics (ICCE)*, 2006.

[87] H. Taghipour, J. Frounchi, and M. H. Zarifi. Design and implementation of MP3 decoder using partial dynamic reconfiguration on Virtex-4 FPGAs. In *Proceedings of International Conference on Computer and Communication Engineering*, 2008.

[88] S. Bouchoux, E. Bourennane, and M. Paindavoine. Implementation of JPEG2000 arithmetic decoder using dynamic reconfiguration of FPGA . In *International Conference on Image Processing (ICIP)*, 2004.

[89] R. Khraisha and J. Lee. A scalable H.264/AVC deblocking filter architecture using dynamic partial reconfiguration. In *Proceedings of IEEE International Conference on Acoustics Speech and Signal Processing (ICASSP)*, 2010.

[90] S. U. Bhandari, S. Subbaraman, S. S. Pujari, and R. Mahajan. Real time video processing on FPGA using on the fly partial reconfiguration. In *International Conference on Signal Processing Systems (ICSPS)*, 2009.

[91] M. Ceschia, M. Violante, M. Sonza Reorda, A. Paccagnella, P. Bernardi, M. Rebaudengo, D. Bortolato, M. Bellato, P. Zambolin, and A. Candelori. Identification and classification of single-event upsets in the configuration memory of SRAM-based FPGAs. *IEEE Transactions on Nuclear Science*, 50(6):2088–2094, Dec. 2003.

[92] C. Bolchini, D. Quarta, and M. D. Santambrogio. SEU mitigation for SRAM-based FPGAs through dynamic partial reconfiguration. In *Proceedings of ACM Great Lakes symposium on VLSI*, 2007.

[93] C. Bolchini, A. Miele, and M. D. Santambrogio. TMR and partial dynamic reconfiguration to mitigate SEU faults in FPGAs. In *Proceedings of IEEE International Symposium on Defect and Fault Tolerance in VLSI Systems (DFT)*, 2007.

[94] J. Heiner, B. Sellers, M. Wirthlin, and J. Kalb. FPGA partial reconfiguration via configuration scrubbing. In *Proceedings of International Conference on Field Programmable Logic and Applications ((FPL)*, 2009.

[95] C. Carmichael. *XAPP216: Correcting Single-Event Upsets Through Virtex Partial Configuration*. Xilinx Inc., June 2000.

[96] C. Carmichael. *XAPP197: Triple Module Redundancy Design Techniques for Virtex FPGAs*. Xilinx Inc., July 2006.

[97] B. Osterloh, H. Michalik, S. A. Habinc, and B. Fiethe. Dynamic partial reconfiguration in space applications. In *NASA/ESA Conference on Adaptive Hardware and Systems*, 2009.

[98] J. W. Lockwood, N. Naufel, J. S. Turner, and D. E. Taylor. Reprogrammable network packet processing on the field programmable port extender (FPX).

In *Proceedings of ACM/SIGDA ninth international symposium on Field programmable gate arrays (FPGA)*, 2001.

[99] D. Yin, D. Unnikrishnan, Y. Liao, L. Gao, and R. Tessier. Customizing virtual networks with partial FPGA reconfiguration. *ACM SIGCOMM Computer Communication Review*, 41(1):57–64, Jan. 2011.

[100] C. Claus, W. Stechele, and A. Herkersdorf. Autovision - a run-time reconfigurable MPSoC architecture for future driver assistance systems. *Information Technology*, 49:181–186, 2007.

[101] W. Gao, K. Kugel, R. Manner, N. Abel, N. Meier, and U. Kebschull. DPR in high energy physics. In *Proceedings of Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2009.

[102] M.J. Wirthlin and B.L. Hutchings. A dynamic insruction set computer. In *In IEEE Symposium on FPGAs for Custom Computing Machines*, 1995.

[103] N.J. Steiner. *Autonomous Computing Systems*. PhD thesis, Virginia Polytechnic Institute and State University, 2008.

[104] J. Torresen, G.A. Senland, and K. Glette. Partial reconfiguration applied in an on-line evolvable pattern recognition system. In *The Nordic Microelectronics event (NORCHIP)*, 2008.

[105] M. Birla and K.N. Vikram. Partial run-time reconfiguration of FPGA for computer vision applications. In *Proceedings of IEEE International Symposium on Parallel and Distributed Processing (IPDPS)*, 2008.

[106] T. Raikovich and B. Fehr. Application of partial reconfiguration of FPGAs in image processing. In *Proceedings of Conference on Ph.D. Research in Microelectronics and Electronics (PRIME)*, 2010.

[107] J. Esquiagola, G. Ozari, M. Teruya, M. Strum, and W. Chau. A dynamically reconfigurable bluetooth base band unit. In *Proceeding of International Conference on Field Programmable Logic and Applications (FPL)*, 2005.

[108] D. Koch and J. Torresen. FPGASort: A high performance sorting architecture exploiting run-time reconfiguration on FPGAs for large problem solving. In *Proceedings of ACM/SIGDA ninth international symposium on Field programmable gate arrays (FPGA)*, 2011.

[109] J. HUANG, M. PARRIS, J. LEE, and R. F. DEMARA. Scalable FPGA-based architecture for DCT computation using dynamic partial reconfiguration. *ACM Transactions on Embedded Computing Systems*, 9:9:1–9:18, 2009.

[110] D. Koch and J. Torresen. Routing optimizations for component-based system design and partial run-time reconfiguration on FPGAs. In *Proceedings of International Conference on Field-Programmable Technology (FPT)*, 2010.

[111] Xilinx Inc. *UG702: Partial Reconfiguration User Guide*, 2010.

[112] P. Schumacher and P. Jha. Fast and accurate resource estimation of RTL-based designs targeting FPGAs. In *Proceedings of Int. Conf. on Field Programmable Logic and Applications (FPL)*, 2008.

[113] LPSolve. Lpsolve reference guide.

[114] M. Liu, Z. Lu, W. Kuehn, S. Yang, and A. Jantsch. A reconfigurable design framework for FPGA adaptive computing. In *International Conference on Reconfigurable Computing and FPGAs (ReConFig)*, 2009.